



Cairo University
Faculty of Graduate Studies for Statistical
Research
Software Engineering Program



Professional Master's Graduation Project

Predicting CI/CD Pipeline Build Failures Using Machine Learning Techniques

Student Name	مؤمن محمد علي حسين
Student ID	٢٠٢٤٠١٦٨١
Program	Master of Software Engineering
Track	Coursework Track
Supervisor Name	د. ماجد ممدوح
Academic Year	[2025 / 2026]
Submission Date	[06/21,2026]

Submitted in partial fulfillment of the requirements for the Professional master's degree in software engineering.

Document Control

Version	Date	Prepared / Updated By	Notes
0.1	May 30, 2026	Moamen Mohamed Aly Hussein	Initial complete draft
0.2	June 5, 2026	Moamen Mohamed Aly Hussein	Incorporated all figures; added appendices and acknowledgments
0.3	June 15, 2026	Moamen Mohamed Aly Hussein	Condensed abstract and introduction; supervisor review revisions
1.0	June 20, 2026	Moamen Mohamed Aly Hussein	Final version submitted for defense

Table of Contents

Abstract

1. Introduction
 2. Problem Definition
 3. Existing Solution Approaches
 4. Proposed Solution
 5. System Analysis and Design
 6. Implementation
 7. Testing and Evaluation
 8. Discussion After Applying the Solution
 9. Conclusion and Future Work
 10. References
-

Abstract

This thesis presents a Hybrid Machine Learning Pipeline for predicting CI/CD pipeline build failures at the moment of commit creation, using only pre-execution features from the GitHub Actions REST API. This addresses a gap in prior work, which has relied on post-execution telemetry that cannot recover the resources a failed build has already consumed.

A dataset of 9,772 real workflow runs was collected from eighteen popular open-source repositories, exhibiting a realistic 89/11 class imbalance. Sixteen features spanning numerical, categorical, binary, and textual (TF-IDF) modalities were engineered, with a 693-token stoplist guarding against identity leakage. Three classifiers (Logistic Regression, Random Forest, XGBoost) were evaluated under a dual-split regime, with an ablation study and a threshold-optimization experiment.

The winning configuration, XGBoost at threshold 0.06, achieves a failure-class F1 of 0.59 (stratified) and 0.62 (chronological), a ROC-AUC of 0.88, and an estimated \$383,000 in annual savings for a mid-sized organization.

Keywords: CI/CD pipelines, build failure prediction, machine learning, hybrid pipeline, TF-IDF, XGBoost, class imbalance, threshold optimization, GitHub Actions.

1. Introduction

1.1 Background

The widespread adoption of continuous integration and continuous deployment (CI/CD) practices has transformed how software is built and released. The contemporary norm is for an automated pipeline to execute on every commit, performing dependency resolution, compilation, testing, security scanning, and frequently a staged deployment, all within minutes of the commit landing in version control. This has produced large gains in software quality, release velocity, and developer productivity.

This automation carries a real cost. Public cloud providers price CI/CD compute by the minute, and the cumulative spend for a large engineering organization can run into millions of dollars per year. Beyond the direct compute cost, every failed build interrupts a developer, consumes engineering attention, and lengthens the lead time from commit to production. The efficiency of CI/CD automation therefore has direct business consequences.

The principal academic studies on CI/CD failure prediction, including Patel's 2019 work, have shown that pipeline outcomes are statistically predictable from features the run itself produces (resource utilization, intermediate step status, build durations, retry behavior). These post-execution approaches are useful for retrospective analysis, but they cannot recover the resources the failing run has already consumed. The natural next question — whether the same outcome can be predicted *before* the run executes, from features available at commit creation — is the central concern of this project.

1.2 Project Motivation

The motivation for this project spans three lenses.

Academic. The existing literature is dominated by post-execution telemetry approaches. These answer a question of limited operational value — they tell us that a build failed, which we already know. A pre-execution approach addresses a question of substantial value: it tells us that a build *will* fail, before we have spent the resources to find out. Demonstrating that this shift can be done with credible accuracy is a contribution to the literature.

Practical. Software organizations spend significant money and engineering attention on CI/CD pipelines that, in aggregate, fail eleven percent of the time. A system that could redirect predicted failures to a lighter pre-flight test, defer them to off-peak compute, or alert the author for review would deliver concrete value. The estimated annual savings for a mid-sized organization (Section 7.4.5) are in the high six figures, while the integration cost is minimal because the system operates as an advisory layer over existing infrastructure.

Personal. The author works professionally as a DevOps and integration engineer with hands-on responsibility for production CI/CD pipelines. The project applies academic machine learning techniques to a real operational problem the author understands from both ends.

1.3 Project Objectives

The project pursues four specific and measurable objectives, each of which is validated against the empirical results reported in Chapter 7.

Objective 1. Build a hybrid machine learning pipeline that combines numerical, categorical, binary, and textual commit-level features into a single end-to-end classifier for pre-execution failure prediction. Success criterion: the pipeline is implemented as a composable scikit-learn pipeline with four parallel feature branches, is fully reproducible from the project repository, and produces predictions in under ten milliseconds per commit on commodity hardware.

Objective 2. Compare three machine learning algorithms (Logistic Regression as linear baseline, Random Forest as ensemble baseline, XGBoost as gradient boosting representative) on the binary failure prediction task under identical conditions, and identify the strongest classifier empirically rather than by assumption. Success criterion: all three classifiers are trained on identical inputs, evaluated under an identical metric suite on both stratified and chronological test sets, and the winning classifier is selected on the basis of an objective decision rule.

Objective 3. Validate the trained model under realistic deployment conditions, including temporal data drift, by reporting metrics on both a stratified random split (primary) and a chronological split (secondary). Success criterion: the winning classifier maintains failure-class F1 within plus or minus three percentage points across the two splits, demonstrating robustness to temporal distribution shift.

Objective 4. Quantify the business impact of the predictive system by translating its precision and recall into estimated annual cost savings under a plausible operational scenario. Success criterion: the analysis produces a quantitative annual savings estimate together with explicit documentation of the assumptions that underlie the estimate, so that readers can adjust it to their own context.

1.4 Project Scope

In scope: The design, implementation, training, evaluation, and documentation of a hybrid machine learning pipeline for pre-execution CI/CD failure prediction. This encompasses real-data collection from the GitHub Actions API, exploratory analysis, feature engineering, the comparative evaluation of three classifiers, an ablation study, a threshold-optimization experiment, and a business-impact estimate. All artefacts (raw and processed data, trained models, results, figures, and full source code) are committed for reproducibility.

Out of scope: The project does not include a live deployment (no HTTP endpoint, CI/CD shim, monitoring dashboard, or online learning); the trained model is a deployment-ready serialized artefact. It also excludes deep learning and transformer-based text encoders — TF-IDF is chosen as a defensible, hardware-independent baseline. Finally, it does not evaluate on corporate proprietary datasets; all data is public open-source, and transfer behaviour is noted as a limitation in Chapter 8. Operationalization and richer text representations are identified as future work.

Assumptions and constraints: The project assumes the GitHub Actions API remains stable, that the business-impact scenario (1,000 daily executions, 11% failure rate, \$75/hour developer rate) is representative of mid-sized organizations, and that the eighteen sampled repositories are sufficiently diverse. It is constrained by the academic timeline, the requirement for reproducibility on commodity hardware without GPU, and the GitHub API rate limits.

1.5 Document Organization

The remainder of this thesis is organized as follows:

- **Chapter 2 (Problem Definition)** articulates the operational problem, its stakeholders, the current as-is process, and the requirements that flow from it.
- **Chapter 3 (Existing Solution Approaches)** surveys prior academic and industrial work across four method families and identifies the limitations the proposed solution addresses.

- **Chapter 4 (Proposed Solution)** describes the Hybrid Machine Learning Pipeline conceptually, including the architecture, technology stack, and principal risks.
- **Chapter 5 (System Analysis and Design)** translates the solution into functional and non-functional requirements, use cases, the data model, and design diagrams.
- **Chapter 6 (Implementation)** documents the realization of the design across the six principal modules and the reproduction instructions.
- **Chapter 7 (Testing and Evaluation)** presents the empirical evaluation: testing strategy, metric suite, classifier comparison, ablation study, threshold optimization, and business-impact analysis, ending with validation of the four objectives.
- **Chapter 8 (Discussion)** reflects on the results, benefits by stakeholder, remaining limitations, and lessons learned.
- **Chapter 9 (Conclusion and Future Work)** summarizes contributions and identifies extensions.

The thesis closes with a **References** section in IEEE format and a set of **Appendices** containing supporting artefacts, code listings, and detailed reproduction instructions.

2. Problem Definition

2.1 Problem Statement

Continuous integration and continuous deployment (CI/CD) pipelines are the standard mechanism by which modern software organizations move code from commit to production. A typical pipeline runs automatically on every commit, executing dependency resolution, compilation, testing, security scanning, and staged deployment. Each stage consumes compute, occupies developer attention, and adds to lead time. When a pipeline succeeds, this investment is well spent; when it fails, the resources committed to the run are largely wasted, and the responsible developer must context-switch back to diagnose and fix the failure.

The problem addressed by this project is the absence of a credible mechanism for predicting whether a pipeline execution will fail *before* it runs. Existing platforms provide rich observability over executions as they occur, but none offers a per-commit failure probability at commit creation. As a result, every commit consumes the full pipeline regardless of risk: a commit introducing a critical syntax error consumes the same compute as one fixing a typo in a comment, even though the former is virtually guaranteed to fail. This uniform treatment is wasteful and represents an unaddressed opportunity for differentiated resource allocation.

The task is therefore to design, build, evaluate, and document a machine learning system that produces a calibrated failure probability for each commit at creation, using only information available at that moment, and to demonstrate that its predictions are accurate enough to support meaningful operational decisions. The challenge is non-trivial because the most predictive features of build outcome (run duration, retry count, resource utilization) are not available until after the build has run — a model using them would describe what happened rather than predict what will happen.

2.2 Stakeholders and Users

Several stakeholder groups are affected by the problem and would benefit from its resolution.

DevOps and platform engineers are responsible for maintaining the CI/CD infrastructure and for the budget that the pipeline compute consumes. They experience the cost of failed builds directly through compute invoices and indirectly through the operational toil of investigating runaway pipeline durations and resource saturation events. A predictive failure system would give them a mechanism to allocate compute resources differentially based on per-commit risk.

Software engineers and individual contributors are responsible for the commits whose builds either succeed or fail. They bear the context-switching cost of a failed build: a typical developer who is interrupted by a build failure mid-task may take ten to twenty minutes to fully re-engage with the failing pipeline, diagnose the root cause, and produce a fix. A predictive system would, in the long run, make their workflows smoother by surfacing high-risk commits earlier and by reducing the volume of after-the-fact debugging.

Engineering managers and tech leads are responsible for the throughput and quality of their teams. They experience pipeline failures as a drag on delivery velocity and as a source of variability in sprint planning. A predictive system would give them an objective signal about commit risk that could inform code-review prioritization, pre-merge approval policies, and team-level health metrics.

Research engineers and machine learning practitioners working on software analytics are interested in the predictive modeling problem itself as a research subject. The dataset, the evaluation protocol, and the trained models produced by this project would all be of value to this audience as a baseline against which alternative approaches can be compared.

Open-source maintainers of the eighteen repositories sampled in this project's dataset are an indirect stakeholder group. The aggregated failure-rate analysis (Figure 7.X) provides them with a comparative view of how their pipeline health benchmarks against peer projects, which has some intrinsic interest even though they did not request the analysis.

2.3 Current Situation / As-Is Process

The current as-is process for handling CI/CD pipeline failures across the surveyed open-source repositories follows a broadly similar pattern, with minor variations by tooling choice. The process can be decomposed into five stages.

In stage one, a developer pushes a commit (or opens a pull request, or merges to a protected branch). The CI/CD provider receives a webhook notification and enqueues a workflow run.

In stage two, the workflow run is dispatched to an available runner, which provisions an ephemeral compute environment, checks out the code at the relevant commit SHA, and begins executing the workflow's defined steps.

In stage three, the workflow steps execute sequentially or in parallel, depending on the workflow definition. Each step consumes compute resources and produces logs that the developer can inspect via the CI/CD provider's user interface.

In stage four, the workflow run reaches a terminal state: success, failure, cancelled, or skipped. If the run fails, the developer is notified via the configured channels (email, Slack, GitHub status check), and the failure is recorded in the run history.

In stage five, the developer reads the failure logs, diagnoses the root cause, modifies the code, and pushes a follow-up commit. The cycle then repeats from stage one for the follow-up commit, with no transfer of information between the failed run and the new one beyond what the developer chooses to apply manually.

The principal weakness of this process, from the perspective of the present project, is that the cost of the failed build (compute resources spent, developer time consumed, calendar time elapsed) is incurred before the developer has any opportunity to act on the information that the build is going to fail. The information about likely failure exists in the commit (its size, its files, its message, its repository context) before the build runs, but the existing infrastructure makes no attempt to extract it.

2.4 Problem Impact

The impact of the problem manifests across several dimensions, each of which has been quantified or estimated in the relevant industry literature.

Compute cost. Public cloud CI/CD providers charge by the minute (GitHub Actions: approximately \$0.008 per minute for standard Linux runners). A typical workflow run for an active open-source repository takes between five and twenty minutes; a failing run

typically consumes a similar amount of compute to a successful one (the failure usually occurs mid-pipeline rather than at the very beginning). At the scale of one thousand pipeline executions per day with an eleven percent failure rate, the direct compute cost of failed runs is approximately \$5,500 per year per organization. While this number is modest in absolute terms, it accumulates linearly with organization size, and it does not include the compute cost of the re-runs that frequently follow a failure.

Developer time cost. A failed build interrupts the developer responsible for the originating commit. The interruption cost has been studied extensively in the software engineering literature: Eyrolle and Cellier (2000) and subsequent work by van der Linden and others place the cost of a single context switch in the range of ten to twenty-three minutes of recovered focus time, depending on the depth of the task that was interrupted. Conservative estimates place the cost at fifteen minutes per failed build. At a fully-loaded developer rate of \$75 per hour, this is \$18.75 per failed build, or approximately \$750,000 per year for an organization at the assumed scale.

Lead time impact. Failed builds extend the lead time from commit to production. Lead time is one of the four key DORA (DevOps Research and Assessment) metrics for engineering organizations and is positively correlated with business outcomes. A pipeline that succeeds on its first attempt has a lead time roughly equal to the workflow duration plus the human review time; a pipeline that fails and requires a re-commit doubles or triples this duration. Across a sufficiently large team, the cumulative lead-time impact of failed builds is substantial.

Cognitive and morale cost. The lived experience of debugging a CI/CD failure that the developer did not anticipate is a source of frustration and a contributor to engineering burnout. The cost of these soft impacts is harder to quantify, but the consistent finding from engineering surveys (Stack Overflow Developer Survey, State of DevOps Report) is that pipeline reliability is among the top reported pain points for working developers.

Aggregate annual cost. Combining the compute, developer-time, and lead-time impacts under the operational scenario documented in Section 7.4.5, the aggregate annual cost of pipeline failures for a mid-sized organization is approximately \$750,000 to \$1,000,000 in directly attributable losses, before considering the soft costs of morale and lead-time impact. A predictive system that meaningfully reduces this loss has clear economic value.

2.5 Requirements Derived from the Problem

The functional and non-functional requirements derived directly from the problem definition above are listed below. These requirements are translated into the detailed system requirements presented in Chapter 5.

High-level functional requirements:

1. The system shall produce a calibrated failure probability for each commit at the moment of commit creation, before any pipeline resources have been consumed.

2. The system shall use only pre-execution features as inputs to its prediction, explicitly excluding any feature whose value is determined by the outcome of the build itself.
3. The system shall consume input features that are readily available from the GitHub Actions REST API (or its equivalent on other CI/CD platforms) without requiring instrumentation changes to existing pipelines.
4. The system shall produce outputs at low enough latency to be invoked synchronously as part of the commit-event handling pipeline (target: under ten milliseconds per prediction).
5. The system shall be evaluated against a real dataset of CI/CD pipeline executions, not against synthetic data, and the evaluation shall report metrics that are appropriate to the class imbalance inherent in the task.

High-level non-functional requirements:

6. The system shall be fully reproducible: any reader of the thesis, given the project repository and a clean Python environment, shall be able to regenerate every reported number to four decimal places.
7. The system shall not depend on proprietary cloud services or paid APIs for its training pipeline, so that the methodology can be adopted by any organization with commodity hardware.
8. The system shall not require GPU compute for either training or inference, so that the operational cost of deployment is negligible.
9. The system shall be documented at a level of detail sufficient to support both academic review and operational adoption by a third party.
10. The system shall protect against identity leakage in its textual features, so that its predictions depend on commit content rather than on contributor identity.

The remainder of the thesis describes how each of these requirements has been addressed by the proposed solution and how the empirical evaluation confirms or qualifies the corresponding claims.

3. Existing Solution Approaches

3.1 Literature and Market Review

The prediction of failures in continuous integration and continuous deployment pipelines is a topic that has attracted sustained academic and industrial attention over the past decade, intersecting the established research areas of defect prediction, software analytics,

and applied machine learning. This section surveys the principal threads of work that bear on the problem, organized into three complementary perspectives: classical defect prediction at the file or class level, build-outcome prediction at the pipeline level, and the more recent line of work that applies natural language processing to commit messages and code reviews as predictive signals.

The classical line of defect prediction work, exemplified by Hassan and Holt (2005) and the subsequent decade of refinements by Kim, Zimmermann, and others, focuses on identifying which files or classes in a codebase are likely to contain bugs based on historical patterns of change frequency, code complexity, and developer activity. This body of work established the methodological foundations that more recent CI/CD failure prediction studies build upon, including the use of cross-validation for evaluation, the careful separation of training and test data along temporal boundaries, and the use of precision-recall metrics in preference to raw accuracy when the failure class is rare. While these classical studies operated at the file granularity rather than the pipeline granularity, the algorithmic toolkit they developed (logistic regression, random forests, gradient boosting) is precisely the toolkit that pipeline failure prediction studies, including the present project, continue to use today.

The build-outcome prediction line of work, which more directly corresponds to the problem addressed by this project, was substantially advanced by Patel’s 2019 study “Research the Use of Machine Learning Models to Predict and Prevent Failures in CI/CD Pipelines and Infrastructure”. Patel proposes a multi-classifier framework that ingests post-execution telemetry from CI/CD runs (resource utilization metrics, build durations, retry counts, and other runtime signals) and produces a probabilistic estimate of whether the corresponding pipeline execution constitutes an anomaly. The study reports promising results on a synthetic dataset and identifies several methodological gaps, in particular the difficulty of obtaining realistic ground-truth data and the challenge of distinguishing transient infrastructure failures from genuine code-level failures. The present project positions itself in direct dialogue with Patel’s framework: it accepts the broad outline of using ensemble machine learning for CI/CD failure prediction, but it shifts the temporal anchor from post-execution to pre-execution. This shift is the central methodological contribution of the project and is what makes the proposed system useful for resource savings, since post-execution prediction can only diagnose failures after the resources have already been spent.

The natural language processing line of work, which has matured rapidly in the past five years, applies textual analysis to commit messages, pull-request descriptions, and code-review comments as auxiliary signals for software engineering tasks. The TravisTorrent dataset (Beller, Gousios, and Zaidman 2017), which aggregates millions of build records from Travis CI across thousands of open-source repositories, has been the standard benchmark for this line of research, and a number of follow-up studies have explored TF-IDF, word embeddings, and more recently transformer-based encoders as representations of the textual signal. The consensus finding from this body of work is that textual features carry non-trivial predictive signal that complements the structured features, but that the magnitude of the complementarity varies widely with the specific task and dataset. The ablation study reported in Chapter 7 of this project contributes a new data point to this

body of evidence by showing that on a fresh GitHub Actions dataset, the TF-IDF text signal in isolation is insufficient to overcome an 8:1 class imbalance, while in combination with the structured features it produces a competitive ranking even when its standalone decision-rule performance is poor.

On the industrial side, several commercial offerings target the CI/CD pipeline reliability problem from different angles. Trunk.io and Buildkite provide analytics dashboards that surface flaky tests and elevated failure rates by repository, branch, or workflow, but they do not produce per-commit predictive estimates. Datadog's CI Visibility product instruments pipeline runs and provides observability over their execution, but again does not predict failures before execution. Recent academic startups such as Aviator and Mergify focus on intelligent merging strategies but do not include predictive failure modeling in their published feature sets. The market gap that the present project addresses is therefore the absence of a documented open-source approach for pre-execution failure prediction that can be deployed alongside (rather than in place of) the existing observability tooling.

3.2 Existing Methods and Techniques

The principal methods used by prior work on CI/CD failure prediction can be grouped into four families, each with distinctive characteristics and limitations.

The first family is **rule-based heuristics**, in which expert engineers define hand-coded rules that flag suspicious commits or builds for additional scrutiny. Typical heuristics include flagging any commit that touches more than fifty files, any commit that arrives outside of business hours, any commit whose message contains the word "hotfix", or any commit on a feature branch with no recent successful build. Rule-based heuristics are easy to explain and easy to deploy, but they are inflexible (each new rule requires manual engineering), they generalize poorly across repositories with different conventions, and they produce binary outputs rather than the calibrated probabilities needed for fine-grained resource-allocation decisions. The present project's structured features (`is_large_commit`, `is_many_files`, `is_off_hours_commit`, `is_weekend_commit`) can be read as the encoding of classical rule-based heuristics into machine-learning features.

The second family is **classical machine learning** with structured features, exemplified by the work of Patel 2019 and similar studies. The typical approach trains a logistic regression, random forest, or gradient boosting classifier on a feature matrix that includes commit size metrics, temporal features, repository metadata, and post-execution telemetry. These methods achieve strong predictive performance and produce interpretable probability outputs, but they generally rely on post-execution features that limit their usefulness for proactive resource savings. The present project belongs to this family in its choice of classifier algorithms but differs in its strict avoidance of post-execution features.

The third family is **natural language processing on commit messages and related artefacts**. This line of work uses TF-IDF, word embeddings such as Word2Vec, or

transformer-based encoders such as BERT to convert commit text into a feature representation that is then fed to a classifier. The reported gains from text features over structured features are typically modest (in the range of two to five percentage points of F1 improvement) but consistent across studies and datasets. The present project includes TF-IDF as a representative of this family and reports its empirical contribution alongside the structured features.

The fourth family is **deep learning with multimodal fusion**, which combines structured tabular features with rich textual or sequential representations through neural network architectures. Studies in this family have reported strong results on large industrial datasets but face significant adoption barriers, including the requirement for GPU compute resources, the loss of interpretability that accompanies deep models, and the difficulty of reproducing the reported numbers across implementations. The present project explicitly considers deep learning as a future-work extension rather than as part of its core contribution, both because the project timeline does not accommodate the additional engineering investment and because the present TF-IDF baseline already provides a useful reference point against which any future deep model would be compared.

3.3 Comparative Analysis

Table 3.1 compares the approaches reviewed in Sections 3.1 and 3.2 against the proposed solution along several dimensions that are relevant to the deployment decision.

Table 3.1 — Comparative analysis of CI/CD failure prediction approaches.

Criterion	Rule-Based Heuristics	Classical ML (post- execution)	NLP- Augmented ML	Deep Learning Multimodal	Proposed Hybrid Pipeline (pre- execution)
Temporal anchor	Pre- execution	Post- execution	Mixed	Mixed	Pre-execution (strict)

Criterion	Rule-Based Heuristics	Classical ML (post-execution)	NLP-Augmented ML	Deep Learning Multimodal	Proposed Hybrid Pipeline (pre-execution)
Resource savings potential	Limited	None	Moderate	High	High
Interpretability	High	High	Moderate	Low	High (feature importances available)
Compute requirements	Negligible	Low (CPU)	Low (CPU)	High (GPU)	Low (CPU)
Reproducibility	High	High	Moderate	Low (stochastic training)	High (fixed seed throughout)
Generalization across repos	Low (per-repo rules)	Moderate	Moderate	Higher	High (eighteen-repository dataset)
Calibrated probability output	No	Yes	Yes	Yes	Yes
Handling of class imbalance	Manual	Standard (weights)	Standard	Standard	Explicit threshold optimization
Empirical headline metric	n/a (not in literature in usable form)	F1 ~0.40-0.60 reported	F1 ~0.45-0.60 reported	F1 ~0.55-0.70 reported on large datasets	F1 = 0.5924 on stratified, 0.6207 on chronological
Deployment friction	Very low	Low	Moderate	High	Low (single joblib file)

Several observations follow from this comparison. The proposed hybrid pipeline matches or exceeds the empirical performance of the classical-ML-with-post-execution-features approach (which serves as the most direct prior baseline) while strictly avoiding post-execution features. It is competitive with the NLP-augmented ML approach despite using only a basic TF-IDF text representation. It falls short of the best deep learning multimodal results reported in the literature on much larger industrial datasets, but it does so at a fraction of the compute cost and with full reproducibility, which are properties of substantial value in academic and small-team production contexts. The combination of pre-execution-only features, fixed-seed reproducibility, dual-split evaluation, and the explicit

threshold-optimization step distinguishes the proposed system from the prior art reviewed in this chapter.

3.4 Limitations of Existing Solutions

The principal limitations of the existing solution approaches, against which the proposed system is positioned, can be summarized as follows.

Limitation 1 — Reliance on post-execution features. The dominant academic approaches to CI/CD failure prediction, including Patel 2019, rely heavily on telemetry that is only available after the build has executed. This makes the predictions useful for retrospective analysis but not for the proactive resource-saving use case that motivates the present project. A pipeline that has already executed has, by definition, already consumed the resources that the prediction was supposed to save.

Limitation 2 — Lack of dual-split robustness reporting. Most published studies report metrics on a single train/test split (typically stratified random or chronological, but rarely both). Single-split reporting can mask substantial performance variability and cannot distinguish a model's intrinsic discriminative ability from its sensitivity to temporal data drift. The dual-split reporting approach adopted by this project provides empirical evidence that the trained classifier maintains its performance under genuine deployment-like conditions.

Limitation 3 — Treatment of decision thresholds as fixed. Most published studies report metrics at the default 0.5 decision threshold, sometimes with a brief note that “threshold tuning could improve results”. The present project elevates threshold optimization to a first-class evaluation phase and demonstrates that for an imbalanced binary classification problem, the gain from threshold optimization can dwarf the gain from feature engineering or hyperparameter tuning. This methodological emphasis is not common in the prior literature on CI/CD failure prediction.

Limitation 4 — Identity leakage through textual features. Studies that include commit messages or related text generally do not document any explicit defense against identity leakage, in which the model learns to recognize author or project identifiers rather than actual content. The 693-token stoplist approach used in this project, while imperfect, represents a more rigorous treatment of this issue than is typical in the literature.

Limitation 5 — Synthetic or proprietary datasets. A substantial fraction of published studies are conducted on synthetic datasets or on proprietary corporate datasets that cannot be independently verified. The present project's use of a freshly collected public GitHub Actions dataset, with the full collection script and resulting CSV committed to the repository, addresses this reproducibility limitation directly.

The proposed solution is designed to address each of these limitations explicitly, and the testing chapter (Chapter 7) provides the empirical evidence that the design choices have the intended effect.

4. Proposed Solution

4.1 Solution Overview

The proposed solution is a Hybrid Machine Learning Pipeline that predicts the outcome (success or failure) of a continuous integration and continuous deployment pipeline execution at the moment a commit is created, using only information that is available before the build runner has executed any user-supplied code. The system takes as input the metadata associated with a commit (repository, workflow name, branch, trigger event, author, lines and files changed) together with the textual commit message itself, and produces as output a real-valued probability that the corresponding workflow run will fail. A configurable decision threshold is then applied to convert this probability into a binary recommendation that downstream systems can act on: a high-confidence failure prediction can be used to skip the full build, route the pipeline to a smaller test subset, or alert the author for review before resources are committed to the execution.

The system is hybrid in two distinct senses. First, it is hybrid at the feature level: the pipeline fuses four parallel feature modalities (numerical, categorical, binary, and textual) through a single ColumnTransformer abstraction that emits a unified sparse feature matrix consumed by a single classifier. This early-fusion design contrasts with late-fusion alternatives in which separate models are trained per modality and their outputs combined at the prediction stage; early fusion was selected here because it allows the classifier to discover interactions between modalities (for example, a particular text token co-occurring with an off-hours commit) that would be invisible to late-fusion ensembles. Second, it is hybrid at the algorithmic level: three candidate classifiers (Logistic Regression, Random Forest, and XGBoost) are trained, evaluated, and compared on the same fused feature matrix, allowing the strongest of the three to be selected empirically rather than by assumption.

The proposed solution complements, rather than replaces, the existing CI/CD validation infrastructure. Builds continue to execute as before; the predictive system serves as an advisory layer that augments developer decision-making with a quantitative risk estimate. Because it is purely advisory and operates entirely on pre-execution data, it introduces no failure mode of its own: if the predictor is unavailable or its prediction is rejected, the underlying pipeline functions exactly as before. This non-intrusive posture is a central design consideration.

4.2 Solution Rationale

The choice of a hybrid early-fusion architecture with three comparative classifiers was deliberate and is defensible against the alternatives considered.

A **single-modality numerical model** would have been simpler but would have ignored the information content of commit messages, which the literature (including Patel 2019) consistently shows contributes to predictive accuracy in software-engineering tasks. A **single-modality textual model** would have ignored the strong structural signals of repository identity, commit size, and authorship; the ablation study (Chapter 7) confirms that text-only achieves only 2.73 percent failure F1, far below structured-only.

A **single-algorithm model** would have removed the empirical evidence needed to justify algorithm selection. The Logistic Regression baseline establishes a linear floor, the Random Forest comparison verifies that boosting performance is not an artefact of one ensemble, and the three-way comparison surfaces algorithm-specific behaviours — notably the dramatic difference in probability calibration discussed in Chapter 7.

A **late-fusion architecture** was rejected because early fusion captures cross-modality interactions (for example, a text token co-occurring with an off-hours commit) that late fusion cannot, and the ColumnTransformer abstraction manages the resulting feature matrix elegantly. A **deep learning approach** was rejected on resource and reproducibility grounds: it would require GPU compute and introduce training-time stochasticity, both outside the project's scope. The deterministic, interpretable TF-IDF baseline runs in seconds on commodity hardware, and transformer-based embeddings are identified as future work.

4.3 Key Features

The proposed solution exposes the following key features to its operational consumers.

The system performs **pre-execution failure prediction**, meaning that it produces a probability estimate at the moment a commit is created, before the workflow run has actually executed. This is the central differentiating feature of the system against the prior art represented by Patel 2019 and similar studies, which rely on post-execution telemetry such as run duration, retry count, and resource utilization. Pre-execution prediction is the property that makes the system useful for resource savings, since it can inform a decision to skip or downscope a build before resources have been committed to it.

The system supports **multi-modal input fusion** through the four-branch ColumnTransformer architecture. The numerical branch processes five engineered features that capture commit size and complexity; the categorical branch processes four features that capture the repository, workflow, branch, and event context; the binary branch passes through six pre-engineered indicator flags; and the textual branch produces a TF-IDF vectorization of the cleaned commit message with up to three thousand unigram and bigram features. The four branches are fused into a single sparse feature matrix and consumed by a single classifier.

The system implements **comparative model evaluation** between Logistic Regression, Random Forest, and XGBoost classifiers. All three are trained on identical inputs and evaluated under an identical metric suite, with the winning classifier selected on the basis of failure-class F1 score under an optimized decision threshold.

The system performs **automatic threshold calibration** through a dedicated optimization stage that sweeps the decision threshold across its full range and selects the operating point that maximizes failure-class F1 (or any alternative criterion such as Youden's J or a business cost function). This calibration step is, as the testing chapter demonstrates, responsible for the largest single performance improvement observed in the project.

The system supports **dual evaluation regimes**, with a stratified random split serving as the primary evaluation and a chronological split serving as a secondary deployment-realism check. This dual reporting goes beyond standard practice in academic machine learning papers and provides empirical evidence that the trained classifier would maintain its performance under genuine production conditions characterized by temporal class drift.

The system is **fully reproducible** from a clean checkout. A fixed random seed of 42 is applied throughout the codebase, all dependencies are pinned to specific versions in the requirements file, and the orchestrator scripts can be executed in sequence to regenerate every model artefact, every metric file, and every figure from the raw collected data. This reproducibility is a deliberate design choice that distinguishes the project from many published machine learning papers, in which the reported results cannot be regenerated by an independent reader.

The system **respects the pre-execution boundary** by explicitly excluding any feature whose value is determined by the outcome of the build itself. The `run_duration_sec`, `run_attempt`, `status`, and `updated_at` columns are dropped before training, and an explicit unit test (TC-002 in the testing chapter) asserts that no post-execution column is present in the final feature set. This boundary enforcement protects the integrity of the project's central claim, which is that useful failure prediction is possible from commit-time information alone.

4.4 Proposed Architecture

The architecture of the Hybrid Machine Learning Pipeline is presented in Figure 4.1. The diagram illustrates the data flow from raw commit metadata at the top, through the four parallel preprocessing branches that constitute the ColumnTransformer, into the fused feature matrix that serves as the unified representation, and finally through the binary classifier (one of Logistic Regression, Random Forest, or XGBoost) that produces the failure probability output. A subsequent decision-threshold step converts this probability into the final success/failure recommendation.

Hybrid Binary-Classification Pipeline Architecture (Real CI/CD Data)



Figure 4.1: Hybrid Machine Learning Pipeline architecture diagram

Figure 4.1: Hybrid Machine Learning Pipeline architecture for binary classification of CI/CD workflow outcomes. Four parallel preprocessing branches (numerical scaling, categorical one-hot encoding, binary passthrough, and TF-IDF text vectorization) are fused via ColumnTransformer into a unified sparse feature matrix, then consumed by one of three comparative binary classifiers.

The architecture is organized around three principles that are worth highlighting explicitly.

First, **separation of concerns**: each preprocessing branch is responsible for exactly one feature modality and produces output that is independent of the other branches. The numerical branch applies StandardScaler; the categorical branch applies OneHotEncoder

with `handle_unknown` set to “ignore” for graceful degradation on unseen categories; the binary branch passes its inputs through unchanged; and the textual branch applies a `TfidfVectorizer` with a unigram-bigram range, a vocabulary cap of three thousand terms, sublinear term-frequency scaling, and a minimum document frequency of five. This branch-level separation makes the architecture easy to extend (a new modality can be added as a fifth branch without touching the others) and easy to ablate (one branch can be removed and the remaining pipeline retrained).

Second, **sparse-matrix efficiency**: the `ColumnTransformer` is configured with a `sparse_threshold` of 0.3, which causes the fused feature matrix to be emitted in SciPy’s Compressed Sparse Row format whenever the proportion of zero entries exceeds seventy percent. Because the TF-IDF branch produces a sparse matrix with thousands of columns, the fused output is overwhelmingly sparse, and using a dense representation would consume tens of gigabytes of memory unnecessarily. Sparse-aware classifiers (Logistic Regression with `liblinear` solver, XGBoost with `hist` tree method) consume the sparse matrix directly without densification, while Random Forest internally converts to dense, which is acceptable for the size of this dataset.

Third, **algorithmic substitutability**: the choice of classifier is encapsulated as the final step of the pipeline, and switching between Logistic Regression, Random Forest, and XGBoost is a one-line change in the orchestrator script. This substitutability is the architectural property that enables the comparative-model evaluation reported in the testing chapter.

4.5 Technology Stack

The technology stack supporting the proposed solution is summarized in Table 4.1. Each component was selected after consideration of alternatives, and the rationale for each selection is briefly noted.

Table 4.1 — Technology stack with selection rationale.

Layer	Technology	Version	Rationale
Language	Python	3.11	De facto standard for applied machine learning; rich library ecosystem; static type hints support code clarity
Operating system	Ubuntu 24.04 LTS (WSL2)	24.04	Reproducible Linux environment matching typical production CI runners

Layer	Technology	Version	Rationale
Data manipulation	pandas	2.2	Mature DataFrame abstraction; excellent CSV I/O performance
Numerical computing	NumPy	1.26	Foundation for vectorized numerical operations; required by all downstream libraries
Classical ML	scikit-learn	1.5	Comprehensive algorithm coverage; ColumnTransformer for hybrid pipelines; mature API
Gradient boosting	XGBoost	2.1	State-of-the-art tree boosting performance; scikit-learn-compatible API; built-in scale_pos_weight for imbalance
Text processing	scikit-learn TfidfVectorizer	1.5	Mature TF-IDF implementation; sparse output for efficiency; configurable n-gram range and vocabulary
HTTP client	Requests	2.32	De facto standard for HTTP in Python; session-based connection pooling
API access	GitHub REST API v3	n/a	Open, well-documented, free at non-trivial rate limits with personal access token
Visualization	Matplotlib + Seaborn	3.9 / 0.13	Publication-quality static figures; full control over typography and layout
Serialization	joblib	1.4	Optimized for NumPy arrays and scikit-learn pipelines; supports compression
Version control	Git	2.43	Universal version-control system; integrates with GitHub
Progress reporting	tqdm	4.66	Lightweight progress bars for long-running collection and training loops

The deliberate decision to use only well-established open-source libraries means that the entire project can be reproduced by any reader with a Python environment, and there is no dependency on proprietary cloud services or paid APIs.

4.6 Risks and Constraints

Several risks and constraints were identified during the design of the proposed solution. Each is documented below together with its mitigation strategy.

Risk 1 — Class imbalance distorts naive metrics. The 89:11 success-to-failure ratio in the dataset means that raw accuracy is a misleading metric and that classifiers trained without imbalance awareness will collapse to a degenerate “always success” prediction. *Mitigation:* the project uses balanced accuracy, failure-class F1, and PR-AUC as the primary evaluation metrics; the classifiers are configured with `class_weight='balanced'` (Logistic

Regression, Random Forest) or `scale_pos_weight` equal to the empirical class ratio (XGBoost); and an explicit threshold-optimization phase calibrates the decision rule to the actual class prior.

Risk 2 — Identity leakage through TF-IDF. Without aggressive text preprocessing, the TF-IDF representation of commit messages would learn to recognize author names and project-specific tokens, producing predictions that depend on contributor identity rather than commit content. *Mitigation:* an algorithmically constructed stoplist of 693 tokens (covering all observed author logins, repository names, and bot signatures) is applied during text cleaning; the `commit_author` column is dropped from the structured features after the `is_bot_author` flag has been derived from it.

Risk 3 — Temporal data drift. The 11 percent overall failure rate in the dataset is not uniformly distributed across the collection window; the chronological-split evaluation reveals a drift to 6.65 percent failure in the most recent twenty percent of the data. A classifier trained on older data may degrade when applied to newer commits. *Mitigation:* the project reports metrics under both stratified and chronological splits, and the threshold optimization is verified to transfer across the two splits within plus or minus three percentage points.

Risk 4 — Limited dataset size. The 9,772-row dataset, while consisting entirely of real production data, is modest compared to industrial-scale datasets that may contain millions of build records. The reported metrics may not generalize precisely to deployments at substantially larger scale. *Mitigation:* the dataset spans eighteen diverse open-source repositories across multiple programming languages and project domains, which provides breadth even if it cannot match the depth of an industrial dataset; the architectural design is fully compatible with larger datasets at no additional engineering cost.

Risk 5 — Reproducibility of API-dependent data collection. The GitHub Actions API returns workflow runs in approximately reverse chronological order with rate limits, so an independent reader running the collection script today would obtain a different (more recent) dataset than the one used to produce the reported results. *Mitigation:* the collected raw dataset is included as a versioned artefact in the project repository, so all downstream analyses can be reproduced exactly even without re-running the collector against the live API.

Constraint 1 — Single-organization dataset. The collected dataset consists entirely of public open-source repositories on GitHub. The model's behavior on proprietary corporate CI/CD pipelines, which may have different commit conventions and different failure-mode distributions, cannot be guaranteed without additional validation on a corporate dataset. This constraint is noted as a future-work item in the conclusion chapter.

Constraint 2 — No live deployment in the project scope. The project produces a trained model and a complete evaluation, but does not include a live API endpoint or a CI/CD integration of the model. Operationalization of the model is straightforward in principle (the trained pipeline is fully serializable and self-contained), but is deliberately out of scope for the academic project and is identified as a future-work item.

5. System Analysis and Design

5.1 Functional Requirements

The functional requirements of the proposed system describe the discrete behaviors that the system must exhibit to fulfill its purpose. Because the proposed solution is a machine learning pipeline rather than a user-facing application, the functional requirements are framed in terms of inputs, transformations, and outputs at well-defined component boundaries, rather than in terms of user interactions.

Table 5.1 — Functional requirements of the Hybrid Machine Learning Pipeline.

ID	Requirement	Priority	Source / Rationale
FR-001	The system shall collect workflow run records from the GitHub Actions REST API given a list of target repositories and a valid personal access token.	High	Source of the real-world data; without this capability, the project cannot exist on a real dataset.
FR-002	The system shall, for each workflow run, retrieve the associated commit message, author, lines added, lines deleted, and files changed via the GitHub commits endpoint.	High	These fields constitute the input features of the predictive model.

ID	Requirement	Priority	Source / Rationale
FR-003	The system shall persist collected raw data to a versioned CSV file on local disk in a format suitable for downstream reproducible analysis.	High	Reproducibility constraint; the collected dataset must remain stable across pipeline reruns.
FR-004	The system shall clean and engineer the collected data into a feature matrix consisting of five numerical, four categorical, six binary, and one textual feature, plus the binary target column.	High	Defines the input shape consumed by the model.
FR-005	The system shall produce two independent train/test partitions of the prepared data: a stratified random split that preserves the class ratio, and a chronological split that uses the most recent twenty percent as the test set.	High	Required by the dual-evaluation strategy described in Chapter 4.
FR-006	The system shall apply an aggressive text cleaning pipeline to commit messages before TF-IDF vectorization, removing URLs, SHA hashes, file paths, PR references, version strings, and all tokens that appear in the project-identifier stoplist.	High	Prevents identity leakage; project's central methodological safeguard.
FR-007	The system shall construct a ColumnTransformer-based hybrid preprocessor with four parallel branches (numerical scaling, categorical one-hot encoding, binary passthrough, textual TF-IDF vectorization) that emits a single sparse feature matrix.	High	Core architectural requirement of the hybrid pipeline.
FR-008	The system shall train three candidate classifiers (Logistic Regression, Random Forest, XGBoost) on the prepared training set using the hybrid preprocessor.	High	Comparative model evaluation is a stated objective.
FR-009	The system shall compute a comprehensive set of evaluation metrics on the test set for each candidate classifier on each split: accuracy, balanced accuracy, precision, recall, F1, ROC-AUC, PR-AUC, and the full confusion matrix.	High	Required to satisfy the reporting expectations of an academic thesis.
FR-010	The system shall perform an ablation study by re-training the winning classifier on three feature configurations (text-only, structured-	High	Required to validate or refute the hybrid claim of the project.

ID	Requirement	Priority	Source / Rationale
	only, hybrid) and reporting the metrics for each.		
FR-011	The system shall perform a threshold optimization sweep on each trained classifier and report the F1-optimal threshold together with the metrics achieved at that threshold.	High	Necessary to obtain useful decisions from probability-calibrated models under heavy imbalance.
FR-012	The system shall compute and report a business-impact estimate (annual cost savings) based on the winning classifier's precision and recall under a documented operational scenario.	Medium	Translates statistical results into deployment value.
FR-013	The system shall produce at least twenty publication-quality figures (300 DPI) covering exploratory data analysis, model architecture, evaluation results, ablation study, threshold optimization, and business impact.	Medium	Required to support the thesis presentation and visual communication of results.
FR-014	The system shall persist every trained classifier to disk using joblib serialization with compression, and shall save a metadata JSON file alongside the model that records its hyperparameters and evaluation metrics.	Medium	Enables reuse of trained models in downstream analysis without re-training.
FR-015	The system shall be fully reproducible from a clean checkout: re-running the orchestrator scripts in sequence with a fixed random seed of 42 shall produce bit-identical metric outputs.	Medium	Strengthens the academic credibility of the reported results.

5.2 Non-Functional Requirements

The non-functional requirements describe quality attributes that the system must satisfy across all of its functional behaviors.

Table 5.2 — Non-functional requirements of the Hybrid Machine Learning Pipeline.

ID	Requirement	Priority	Source / Rationale
NFR-001	The full pipeline execution (data preparation through threshold optimization) shall complete within thirty minutes on commodity hardware (modern laptop with 16 GB RAM, no GPU).	High	Reproducibility for thesis evaluators; avoids dependence on specialized hardware.

ID	Requirement	Priority	Source / Rationale
NFR-002	The inference latency of the winning classifier on a single commit shall not exceed ten milliseconds on the same hardware.	High	Required for any future operational deployment in which the model would be invoked at commit time.
NFR-003	The trained model artefact size shall not exceed ten megabytes per classifier when serialized with joblib compression level three.	Medium	Keeps the repository manageable and facilitates model versioning.
NFR-004	The codebase shall use Python type hints throughout, with no untyped function signatures in the src/ directory.	Medium	Maintainability and self-documentation.
NFR-005	The codebase shall follow PEP-8 style conventions, enforced through a linter pass before any phase is marked complete.	Medium	Code quality and readability.
NFR-006	The system shall not embed any secrets (API tokens, credentials) in source files; all secrets shall be consumed from environment variables.	High	Standard security practice.
NFR-007	The pre-execution boundary shall be enforced by an automated test that fails if any post-execution column (run_duration_sec, run_attempt, status, updated_at) is present in the training feature matrix.	High	Protects the central scientific claim of the project.
NFR-008	All figures shall be saved at 300 DPI and shall use a consistent color palette, font family, and grid style across the entire thesis.	Medium	Publication quality.
NFR-009	All numeric results reported in the thesis (tables, figures, prose) shall be traceable to a specific JSON file in the results/ directory, which is generated by the orchestrator scripts and committed alongside the code.	High	Reproducibility and audit.
NFR-010	The dataset collection script shall respect the GitHub API rate limit and shall pause-and-resume gracefully when the limit is approached, without losing already-collected data.	High	Operational requirement for the multi-hour collection phase.

5.3 Use Cases

Although the system is a machine learning pipeline rather than an end-user application, three principal use cases describe how the system would be exercised by the actors who interact with it.

Use Case UC-1 — Researcher reproduces the full pipeline from scratch. *Actor:* A reader of the thesis (academic supervisor, evaluator, or independent researcher) who wishes to reproduce the reported results from a clean environment. *Preconditions:* A Linux or macOS system with Python 3.11 installed; a valid GitHub personal access token for the data collection phase. *Main flow:* The actor clones the repository, creates a Python virtual environment, installs dependencies via `pip install -r requirements.txt`, sets the `GITHUB_TOKEN` environment variable, and executes the phase orchestrator scripts in order: `run_phase0.py` through `run_phase5.py`. *Alternative flow:* The actor skips `run_phase0.py` (which requires roughly two hours due to API rate limits) and uses the pre-collected dataset committed to `data/raw/github_actions_real.csv`. *Postconditions:* The `data/processed/`, `models/`, `results/`, and `figs/` directories are populated with outputs that match those reported in the thesis to four decimal places.

Use Case UC-2 — Practitioner uses the trained model for new prediction. *Actor:* A DevOps engineer or research engineer who has a trained classifier and wishes to score a new commit. *Preconditions:* The trained model file (`models/best_optimized.joblib`) and the data preparation module are available. *Main flow:* The actor constructs a single-row DataFrame containing the required input features (repository, workflow name, branch, event, lines added, lines deleted, files changed, commit message, and so on), passes it through the data preparation function to produce a feature row in the model's expected schema, calls `pipeline.predict_proba(X)[0, 1]` to obtain the failure probability, and compares it against the optimized threshold of 0.06. *Postconditions:* The actor receives a real-valued failure probability and a binary recommendation that can be passed to downstream tooling.

Use Case UC-3 — Researcher extends the system with a new classifier or new feature. *Actor:* A researcher who wishes to evaluate a new classifier (for example, LightGBM, CatBoost, or a transformer-based text encoder) against the existing baselines. *Preconditions:* The full project codebase is checked out. *Main flow:* The actor adds a new pipeline factory function to `src/hybrid_pipeline.py` following the pattern established by the existing three factories; adds the new pipeline to the `get_all_pipelines` registry; and re-runs `run_phase4.py`. The evaluation framework automatically includes the new pipeline in the comparison tables and figures. *Postconditions:* The new classifier appears in all evaluation outputs alongside the existing three baselines, allowing apples-to-apples comparison.

5.4 Data Model

The data model of the project consists of a sequence of structured artefacts that flow from raw API responses to fully prepared model inputs. The schema of each artefact is presented in Table 5.3.

Table 5.3 — Data artefact schema across pipeline phases.

Phase	Artefact	Rows	Key Columns
Raw collection	data/raw/github_actions_real.csv	9,772	run_id, repository, workflow_name, event, branch, conclusion, status, created_at, updated_at, run_duration_sec, run_attempt, commit_sha, commit_message, commit_author, lines_added, lines_deleted, total_changes, files_changed, commit_date
Cleaned & engineered	data/processed/cicd_prepared.csv	9,772	repository, workflow_name, branch, event, commit_message, commit_message_clean, log_lines_added, log_lines_deleted, log_files_changed, commit_message_length, avg_lines_per_file, is_large_commit, is_many_files, is_weekend_commit, is_off_hours_commit, is_bot_author, was_truncated, conclusion (binary), commit_date
Stratified train	data/processed/train_stratified.csv	7,817	(same as cicd_prepared.csv)

Phase	Artefact	Rows	Key Columns
Stratified test	data/processed/test_stratified.csv	1,955	(same as cisd_prepared.csv)
Chronological train	data/processed/train_chronological.csv	7,817	(same as cisd_prepared.csv)
Chronological test	data/processed/test_chronological.csv	1,955	(same as cisd_prepared.csv)

The transformations from one artefact to the next are deterministic given the random seed, and the schema of each artefact is documented at the top of the corresponding source module. The data flow is illustrated conceptually in Figure 5.1.

The data flow from raw GitHub Actions API response through prepared training and test sets is visualized in the project repository under `figs/fig_data_flow.png` and is summarized verbally as follows: each raw workflow run record is parsed from JSON into a flat CSV row, then enriched with commit-level statistics from a second API call; the resulting rows are subjected to cleaning, feature engineering, high-cardinality bucketing, and aggressive text normalization; the cleaned data is then partitioned into stratified and chronological splits and persisted as four CSV files that downstream phases consume directly.

The model itself does not maintain any persistent state beyond the serialized pipeline artefact; there is no database, no in-memory cache, and no inter-session state. Each prediction is computed deterministically from the input features and the loaded model.

5.5 User Interface Design

Because the proposed solution is an offline analytical pipeline rather than an interactive application, there is no graphical user interface in the conventional sense. The “user interface” of the system is the command-line interface exposed by the six phase orchestrator scripts. Each script accepts no positional arguments and reads its configuration from module-level constants in the corresponding source files; this design favors reproducibility (every run uses identical configuration) over interactive flexibility.

The user-facing output of the system consists of three distinct artefact categories. The first is the set of structured result files under `results/` (JSON files with one or more metric tables per phase). The second is the set of publication-quality figures under `figs/` (twenty-one PNG files at 300 DPI). The third is the set of trained model files under `models/` (joblib serializations of fitted scikit-learn pipelines). All three artefact categories are intended to be consumed directly by human readers (thesis evaluators, research engineers reviewing the work) rather than by downstream automated systems.

For the purpose of a future production deployment, the natural user interface extension would be a thin HTTP service that wraps the trained pipeline and exposes a single endpoint

accepting a commit metadata payload and returning a failure probability. This deployment surface is explicitly identified as a future-work item in Chapter 9.

5.6 System Design Diagrams

Three system design diagrams capture the design of the proposed solution at different levels of abstraction.

The **component diagram** in Figure 5.2 shows the project codebase decomposed into its principal modules and the dependencies between them. The diagram identifies seven modules (data collection, EDA, data preparation, hybrid pipeline, training and evaluation, threshold optimization, visualization) plus the corresponding orchestrator scripts.

The project codebase decomposes into seven principal modules located under src/: collect_github_data.py (data ingestion via the GitHub REST API), eda.py (exploratory data analysis and chart generation), data_preparation.py (cleaning, feature engineering, and split production), hybrid_pipeline.py (the four-branch ColumnTransformer architecture and three classifier factories), train_evaluate.py (full training, metric computation, and ablation study), threshold_optimization.py (decision-threshold sweep and selection), and visualization.py (the ThesisPlotter class that enforces consistent figure styling). Each module is imported by the corresponding phase orchestrator script (run_phase0.py through run_phase5.py).

The **data flow diagram** in Figure 5.3 traces the path of a single commit through the system, from the raw API response through cleaning, feature engineering, vectorization, and into the final prediction. This diagram is the operational complement to the architectural diagram in Figure 4.1: where Figure 4.1 shows the system in its training configuration, Figure 5.3 shows the system in its inference configuration.

The inference data flow for a single commit proceeds as follows: the input record (a single-row pandas DataFrame containing the seventeen feature columns) is passed to the loaded pipeline's predict_proba method; the ColumnTransformer applies the four branch transformations in parallel and produces a 3,090-column sparse feature vector; the trained XGBoost classifier produces a real-valued probability in the range [0, 1]; the optimized threshold of 0.06 is applied to produce a binary success/failure recommendation. The entire flow executes in under one millisecond on commodity hardware.

The **sequence diagram** in Figure 5.4 captures the order of operations performed by the full pipeline orchestration. The sequence begins with the actor invoking run_phase0.py and ends with the production of the final business-impact JSON file by run_phase5.py. Intermediate steps are clearly numbered to facilitate troubleshooting in the event of a pipeline failure.

The pipeline orchestration proceeds through six sequential phases: (1) Phase 0 collects raw data from the GitHub Actions API and persists it to data/raw/; (2) Phase 1 performs exploratory analysis and generates six diagnostic figures; (3) Phase 2 cleans the raw data, engineers sixteen features, and produces both stratified and chronological splits; (4) Phase 3 builds the hybrid pipeline architecture and validates it on a 1,000-row sample; (5) Phase 4

trains all three classifiers on the full data, evaluates them on both splits, runs the ablation study, and produces seven evaluation figures; (6) Phase 5 performs threshold optimization and produces two final figures plus the business-impact estimate. Each phase writes its outputs to deterministic locations and can be re-executed independently.

6. Implementation

6.1 Development Environment

The implementation of the proposed CI/CD failure prediction system was carried out in a reproducible Linux-based development environment that mirrors the production conditions under which the model would eventually be deployed. All development activities were conducted on Ubuntu 24.04 LTS running under the Windows Subsystem for Linux (WSL2), with Python 3.11 as the primary programming language and Visual Studio Code as the integrated development environment. Version control was managed through Git, with the project organized as a single repository following a clean, modular directory structure that separates raw data, processed data, source code, trained models, results, figures, and experimental logs into distinct top-level directories.

The technical stack was selected to balance scientific rigor, community support, and reproducibility. The core data manipulation layer was built on top of pandas (version 2.2) and NumPy (version 1.26), which together provided the foundation for all dataset operations including loading, cleaning, feature engineering, and split generation. The machine learning components were implemented using scikit-learn (version 1.5) for the Logistic Regression and Random Forest classifiers, the ColumnTransformer abstraction, the TF-IDF vectorizer, and all evaluation metrics. The gradient boosting model was provided by XGBoost (version 2.1), which offers a scikit-learn compatible API and integrates seamlessly with the rest of the pipeline. Visualization was handled by Matplotlib (version 3.9) and Seaborn (version 0.13), with all figures produced at 300 dots per inch (DPI) to ensure publication-quality output. Serialization of trained models was managed through joblib with compression level three, which keeps model artefacts compact while preserving full deserialization fidelity.

Data collection from the GitHub Actions API was performed using the Requests library (version 2.32) with a persistent HTTP session, augmented by the tqdm library for real-time progress feedback during the multi-hour collection runs. All API calls were authenticated using a personal access token with read-only scope on public repositories, and the collector implemented exponential backoff together with automatic rate-limit detection to handle the GitHub API constraints gracefully. The complete list of dependencies, together with their pinned versions, is captured in the project's requirements.txt file for full reproducibility.

6.2 Implementation Details

The implementation is organized as a sequence of clearly delineated phases, each corresponding to a self-contained Python module under the `src/` directory. The orchestration of each phase is handled by a dedicated runner script, which allows individual phases to be re-executed in isolation without rebuilding the entire pipeline. This modular design proved essential during the iterative development process, where individual components were refined multiple times in response to empirical observations.

6.2.1 Data Collection Module

The data collection module, implemented in `src/collect_github_data.py`, is responsible for retrieving real-world CI/CD pipeline data from the GitHub Actions API. The collector accepts a curated list of eighteen high-activity open-source repositories spanning multiple programming languages and project domains, including `facebook/react`, `microsoft/vscode`, `tensorflow/tensorflow`, `pytorch/pytorch`, `rust-lang/rust`, `python/cpython`, and `elastic/elasticsearch`. For each repository, the collector performs paginated calls to the workflow runs endpoint and, for each workflow run, follows up with a second call to the commits endpoint to retrieve the associated commit message, author, and code change statistics.

The collector applies several quality filters at ingestion time to ensure that only meaningful records enter the dataset. Workflow runs are retained only if their status is “completed” and their conclusion is one of {“success”, “failure”}; cancelled, skipped, and neutral runs are discarded because they do not represent genuine outcomes of the build process. Commit messages that are empty or shorter than five characters are filtered out, and messages longer than one thousand characters are truncated to that length to bound the memory footprint of the downstream text features. The collector also implements incremental writes to disk, appending each batch of completed records to the master CSV file so that an interruption mid-collection does not result in data loss.

The final raw dataset, stored at `data/raw/github_actions_real.csv`, contains 9,772 unique workflow runs collected over approximately 122 minutes of wall-clock time, with two automatic rate-limit pauses absorbed by the collector’s backoff logic. The resulting class distribution of 89.0 percent successes and 11.0 percent failures reflects the natural imbalance observed in mature open-source CI/CD environments and constitutes a realistic foundation for the predictive modeling task.

6.2.2 Data Preparation Module

The data preparation module, implemented in `src/data_preparation.py`, transforms the raw collected data into a model-ready feature matrix. This module is the single source of truth for the project’s feature definitions and is imported by every subsequent component, ensuring that the training pipeline, the evaluation harness, and the deployment-ready inference logic all operate on an identical feature schema.

Three classes of transformations are applied in sequence. The first stage performs basic cleaning: rows with null commit messages are dropped, the `commit_author` field is filled

with the literal string “unknown” for the small fraction of records where it is missing, and the author_association column is dropped entirely because it returned as fully null from the commits endpoint. The second stage performs feature engineering, deriving sixteen new features from the raw inputs. These derived features capture commit-size signals through logarithmic transformations of the raw line and file counts, temporal signals through indicators for weekend and off-hours commits, structural signals through ratios such as average lines per file, and behavioral signals through binary flags such as is_bot_author, which is computed by pattern-matching the author login against known bot signatures (Dependabot, Renovate, bors, github-actions). The third stage performs cardinality reduction on the high-cardinality categorical columns: workflow_name (623 unique values) and branch (1,396 unique values) are bucketed by retaining the top twenty most frequent values and aggregating the remainder into an “other” category, reducing each column to twenty-one and sixteen distinct values respectively. This step is essential because naive one-hot encoding of these columns would have produced thousands of sparse indicator features and would have made the model brittle to any unseen value at inference time.

A separate function, clean_commit_message_for_nlp, is responsible for preparing the textual content for downstream TF-IDF vectorization. This function applies an aggressive regular-expression pipeline that strips URLs, email addresses, SHA hashes of both eight and forty character lengths, pull-request and issue references in the GitHub-specific formats (#123, gh-12345, (#4567)), version strings such as 1.2.3 and v2.0, file paths, and the standard “Co-authored-by” and “Signed-off-by” trailers. After regex-based cleaning, a token-level filter removes any word that matches an entry in an algorithmically constructed stoplist of 693 tokens. This stoplist is built by scanning the entire collected dataset and extracting all unique author logins (558 entries), all organization and repository names (eighteen tokens), and a manually curated set of common bot signatures and CI-specific noise tokens. The aggressive cleaning is necessary because preliminary experiments revealed that without it, the TF-IDF features would learn project-specific and author-specific identifiers rather than genuine signals about commit content, which would amount to a form of identity leakage and would inflate model performance on patterns that do not generalize to unseen repositories.



Figure 6.1: Discriminative vocabulary after text cleaning

Figure 6.1: Word clouds showing the most discriminative tokens for the failure class (left) and the success class (right) after the aggressive text cleaning and stoplist filtering described above. The failure-class vocabulary now contains infrastructure-related terms (timezone, utc, thresholds, inodesfree, stderr, borrow) rather than the author names and project identifiers that dominated before cleaning.

The preparation module also implements the two split strategies adopted by the project. The `stratified_split` function performs a random 80/20 partition that preserves the class ratio exactly, yielding training and test sets with failure rates of 10.98 percent and 10.95 percent respectively. The `chronological_split` function sorts the data by commit timestamp and uses the most recent twenty percent as the test set, which yields a training failure rate of 12.05 percent and a test failure rate of 6.65 percent, reflecting the real temporal drift observed in the underlying repositories as their continuous integration practices matured. Both split strategies are run during preparation, producing four CSV files under `data/processed/` that downstream phases can consume directly.

6.2.3 Hybrid Pipeline Module

The hybrid pipeline module, implemented in `src/hybrid_pipeline.py`, defines the model architecture as a composable scikit-learn pipeline. At the heart of this module is the `build_preprocessor` function, which constructs a single `ColumnTransformer` with four parallel branches. The numerical branch applies `StandardScaler` to the five engineered numerical features (`log_lines_added`, `log_lines_deleted`, `log_files_changed`, `commit_message_length`, `avg_lines_per_file`). The categorical branch applies `OneHotEncoder` with `handle_unknown` set to “ignore” to the four bucketed categorical features (`repository`, `workflow_name`, `branch`, `event`), so that any unseen category at inference time is silently mapped to an all-zero vector rather than raising an exception. The binary branch passes the six pre-engineered binary indicators through unchanged (`is_large_commit`, `is_many_files`, `is_weekend_commit`, `is_off_hours_commit`, `is_bot_author`, `was_truncated`). The text branch applies a `TfidfVectorizer` configured with a maximum vocabulary of three thousand terms, a unigram-plus-bigram range, a minimum document frequency of five, a maximum document frequency of 0.95, sublinear term-frequency scaling, and lowercase normalization. The `ColumnTransformer` is instantiated with `sparse_threshold` equal to 0.3 and `n_jobs` equal to negative one, which enables parallel transformation across all four branches and emits the combined feature matrix as a sparse SciPy CSR matrix.

Three classifier factory functions wrap the preprocessor into complete end-to-end pipelines. The Logistic Regression pipeline uses the `liblinear` solver with `class_weight` set to “balanced” and a maximum of one thousand iterations; this combination handles the 8:1 class imbalance through automatic class re-weighting and is well suited to the high-dimensional sparse feature matrix produced by the TF-IDF branch. The Random Forest pipeline uses two hundred trees with a maximum depth of twenty-five, minimum samples per split of ten, minimum samples per leaf of four, balanced class weights, and parallel execution across all available CPU cores. The XGBoost pipeline uses three hundred boosting rounds with a learning rate of 0.1, a maximum tree depth of eight, subsample ratio of 0.85 on both rows and columns, L1 regularization of 0.1, L2 regularization of 1.0, the histogram-

based tree construction algorithm for speed, and a `scale_pos_weight` of 8.12, which is the empirically derived ratio of the negative to positive class counts in the training set. Because XGBoost's binary classification interface requires integer-encoded labels, a thin wrapper class named `LabelEncoderForBinary` intercepts the `fit` and `predict` calls to perform transparent label encoding and decoding, allowing the rest of the codebase to operate on the human-readable conclusion strings without complication.

All three pipelines expose the standard scikit-learn `fit`, `predict`, and `predict_proba` interfaces, and all three are fully serializable via `joblib` without any custom pickle protocols. The module also exposes a `get_all_pipelines` helper that returns a dictionary of the three pipelines in canonical order, which is used by the evaluation module to iterate over the candidate models uniformly.

6.2.4 Training and Evaluation Module

The training and evaluation module, implemented in `src/train_evaluate.py`, is responsible for fitting each candidate model on the training data, generating predictions on the test data, and computing a comprehensive set of evaluation metrics. The module operates on two evaluation regimes in parallel: the stratified split serves as the primary evaluation and yields the headline metrics reported in the results chapter, while the chronological split serves as a secondary evaluation that tests the robustness of the trained models against the temporal drift observed in the underlying data distribution. Each trained model is persisted to the `models/` directory with `joblib` compression, allowing later phases (in particular the threshold optimization stage) to load the trained estimators without re-training.

The metric computation routine produces a full set of binary classification scores for each model on each split, including overall accuracy, balanced accuracy, precision, recall, and F1 score on the failure class (treated as the positive class), macro-averaged F1, weighted F1, ROC-AUC, precision-recall AUC, specificity, and the full confusion matrix. The dual reporting of accuracy and balanced accuracy is intentional and reflects best practice for imbalanced binary classification: under an 89:11 class prior, a degenerate “always predict success” classifier would already achieve 89 percent raw accuracy, so balanced accuracy and the failure-class F1 are the metrics on which model selection and the headline results are based.

An ablation study function, `run_ablation_study`, isolates the contribution of the textual modality by re-fitting the XGBoost classifier under three modified configurations: text-only (using only the `TfidfVectorizer` branch), structured-only (using only the numerical, categorical, and binary branches), and the full hybrid configuration. The same hyperparameters, random seed, and training data are used across all three configurations, ensuring that any performance differences are attributable solely to the change in feature composition. The ablation results are written to `results/ablation_study.json` for inclusion in the testing and evaluation chapter.

6.2.5 Threshold Optimization Module

The threshold optimization module, implemented in `src/threshold_optimization.py`, addresses a subtle but important issue that surfaced during the initial evaluation pass. The default decision threshold of 0.5 used by scikit-learn classifiers is calibrated for a balanced class prior and is poorly suited to the 8:1 imbalance present in this dataset, particularly for

the XGBoost classifier whose probability outputs are skewed toward the majority class even after the application of `scale_pos_weight`. The module implements a threshold sweep from 0.05 to 0.95 in increments of 0.01, computing the failure-class F1, Youden's J statistic, balanced accuracy, and a business-cost objective at each threshold value. The threshold that maximizes each metric is reported, and the optimal F1 threshold is then applied to both the stratified and chronological test sets to verify that the calibration transfers cleanly across splits.

For the winning XGBoost classifier, the F1-optimal threshold was found to be 0.06 rather than the default 0.5, an order-of-magnitude shift that reflects the underlying class imbalance. Applying this optimized threshold lifted the failure-class F1 from 32.2 percent to 59.2 percent on the stratified test set, with the recall rising from 20.1 percent to 62.2 percent. The same threshold applied to the chronological test set produced a failure-class F1 of 62.1 percent, confirming that the optimization is not a stratified-split artefact but a genuine improvement that transfers to deployment-like conditions.

6.2.6 Visualization Module

The visualization module, implemented in `src/visualization.py`, encapsulates a single class named `ThesisPlotter` that centralizes the styling of every figure produced by the project. The class enforces a consistent academic aesthetic across all twenty-one publication-quality figures: a serif font family with Computer Modern as the primary face, a fixed color palette designed for both color and grayscale reproduction, a uniform tick and grid style, and an automatic save routine that writes every figure at 300 DPI in PNG format and appends a thesis-ready caption to `figs/captions.md`. Centralizing the styling in a single class proved valuable both for consistency, since every figure across the nine phases of the project shares the same visual language, and for maintainability, since global style adjustments require changes in only one place.

6.3 Important Code and Configuration Decisions

Several implementation decisions had a material impact on the quality of the final results and warrant explicit discussion. These decisions emerged through iterative experimentation and represent course corrections that were applied as empirical observations contradicted the initial design assumptions.

The first decision concerned the choice of dataset. The project was initially attempted on a publicly available synthetic CI/CD failure logs dataset of forty-five thousand rows. While that dataset offered the convenience of a ready-made schema with explicit failure stage and severity columns, preliminary exploratory analysis revealed that the columns were statistically independent of one another and that the textual error messages consisted of random character strings rather than meaningful text. No machine learning model can extract signal from independent columns of noise, and so the project was redirected toward collecting a genuine dataset from the GitHub Actions API. This shift cost approximately two hours of additional engineering effort and required rewriting the data

preparation logic, but it transformed the project from an exercise in pattern fabrication into a credible piece of applied machine learning research.

The second decision concerned the boundary between pre-execution and post-execution features. CI/CD pipelines naturally produce a rich stream of features over the course of a build, including run duration, retry count, CPU and memory utilization, and exit codes. From a modeling perspective these features are tempting because they correlate strongly with the outcome. From the deployment perspective adopted by this project, however, they are post-execution features and using them would amount to data leakage, since they are not available at the moment a commit is created. The implementation therefore explicitly drops `run_duration_sec`, `run_attempt`, the engineered `is_retry` flag derived from it, `status`, and `updated_at` from the feature set, and all evaluation is performed strictly on pre-execution features available at commit time.

The third decision concerned text preprocessing. The first iteration of the TF-IDF pipeline used only the default scikit-learn stopword list and produced a discriminative vocabulary that was dominated by author names (“anna”, “kamat”, “yagiz”, “trivikr”) and project-specific identifiers (“ractors”, “callcache”, “ifrt”). This is a form of identity leakage in which the model learns to recognize which contributor or project a commit belongs to, rather than learning about the content of the commit itself. The aggressive regex cleaning and the algorithmically constructed 693-token stoplist described in Section 6.2.2 were introduced specifically to address this issue, and they reduced the project-identity leakage substantially although they did not eliminate it entirely. The residual leakage is honestly reported and discussed in the testing and evaluation chapter.

The fourth decision concerned the evaluation strategy. The initial plan called for a single chronological train/test split, on the grounds that this best simulates a production deployment in which a model trained on historical data is applied to future commits. However, the chronological split exhibited a substantial class drift, with the test failure rate of 6.65 percent being almost half the training failure rate of 12.05 percent. This drift makes raw accuracy on the chronological test set difficult to interpret, since a degenerate “always predict success” classifier would already achieve 93.4 percent accuracy. The project therefore adopted a dual-split strategy in which the stratified random split serves as the primary evaluation and the chronological split serves as a secondary robustness check; this is the standard contemporary practice in applied machine learning research on imbalanced temporally structured data.

The fifth decision concerned the handling of XGBoost’s classification threshold. The initial evaluation pass treated the default 0.5 threshold as fixed, which led to an apparent conclusion that XGBoost was a poor classifier despite achieving the highest ROC-AUC and PR-AUC of all candidates. A focused investigation revealed that the model’s probability

outputs were well-calibrated for ranking but were systematically biased toward the majority class for hard decisions. A dedicated threshold optimization phase was added to the pipeline, and it produced the single largest performance improvement in the project: a 27-percentage-point gain in failure-class F1 from a one-line code change. This experience reinforces a methodological lesson that the testing and evaluation chapter discusses in detail, namely that probability calibration and decision-rule selection are first-class concerns for imbalanced binary classification, not afterthoughts.

6.4 Security and Data Protection

Although the project does not deploy a user-facing system and does not handle personally identifiable information in the conventional sense, several security and data-protection considerations were addressed during implementation. The GitHub personal access token used by the data collector was scoped to read-only access on public repositories and was rotated immediately after the data collection phase completed, with no token value committed to the repository or embedded in any source file. The collector script consumes the token strictly through the `GITHUB_TOKEN` environment variable, which is the recommended pattern for handling secrets in shell-driven workflows.

The collected dataset includes the GitHub login of each commit author, which is publicly available information but nonetheless constitutes a weak personal identifier. To prevent the model from learning author-specific patterns that would not generalize across organizations or projects, the `commit_author` column is dropped from the feature set after the `is_bot_author` flag has been derived from it. The 693-token stoplist used in text cleaning includes every author login observed in the dataset, ensuring that no author name can leak into the model through the TF-IDF branch. Together, these two measures reduce the risk that the trained model would behave as an inadvertent author classifier rather than as a content-based failure predictor.

All trained models are stored locally and are not transmitted to any external service. The full reproducibility of the pipeline, from raw API responses through to final predictions, means that the trained models can always be regenerated from the saved raw data; the model artefacts themselves are therefore not single points of failure for the project, and no special protections are applied to them beyond standard file-system permissions.

6.5 Deployment and Execution Instructions

The project is fully reproducible from a clean checkout in under one hour, excluding the GitHub data collection step, which takes approximately two hours due to API rate limits. The recommended environment is Python 3.11 on Linux or macOS, with a virtual environment created in the project root via `python -m venv .venv` and dependencies installed via `pip install -r requirements.txt`. Once the environment is configured, the full pipeline can be reproduced by running the six phase orchestrator scripts in order: `run_phase0.py` for data collection (which requires the `GITHUB_TOKEN` environment variable to be set), `run_phase1.py` for exploratory data analysis, `run_phase2.py` and `run_phase2_5.py` for data preparation and quality refinement, `run_phase3.py` for pipeline

architecture validation, `run_phase4.py` for full training and evaluation, and `run_phase5.py` for threshold optimization. Each phase writes its outputs to deterministic locations under `data/`, `models/`, `results/`, and `figs/`, and each can be re-run independently without invalidating the outputs of unrelated phases. A fixed random seed of 42 is applied throughout the codebase, so two runs of the same phase on the same input data produce bit-identical outputs.

7. Testing and Evaluation

7.1 Testing Strategy

The evaluation of the proposed CI/CD failure prediction system follows a multi-layered strategy designed to assess both the statistical performance of the classifier and the robustness of that performance under realistic deployment conditions. Unlike a conventional software system, which is verified through unit tests, integration tests, and acceptance tests against deterministic specifications, a machine learning system must be evaluated against a held-out portion of real data using statistical metrics that capture the trade-offs inherent in probabilistic prediction under class imbalance. The evaluation strategy adopted by this project rests on five complementary components, each of which addresses a distinct aspect of model quality.

The first component is the dual-split evaluation regime. As described in the implementation chapter, the prepared dataset is partitioned in two independent ways: a stratified random split that preserves the 89:11 class ratio across train and test, and a chronological split that places the most recent twenty percent of commits in the test set. The stratified split serves as the primary evaluation because it isolates the classifier's intrinsic discriminative ability from any confound introduced by temporal distribution shift, while the chronological split serves as a secondary deployment-realism evaluation that quantifies how much, if at all, the model degrades when applied to commits drawn from a time period it never saw during training. Reporting both is a deliberate design choice that gives the thesis defense panel an honest view of how the model would behave in a real production environment.

The second component is the comparative-model evaluation, in which three candidate classifiers—Logistic Regression as a linear baseline, Random Forest as a non-linear ensemble baseline, and XGBoost as a gradient boosting state-of-the-art representative—are trained and tested on identical inputs under identical hyperparameter regimes (within the constraints of each algorithm). This three-way comparison allows the project to identify

the strongest classifier empirically rather than by assumption, and it surfaces algorithm-specific behaviors that would not be visible from any single model in isolation.

The third component is the ablation study. The hybrid pipeline combines four feature modalities (numerical, categorical, binary, and textual), and a central claim of the project is that the combination of these modalities is more predictive than any one of them alone. This claim cannot be validated by simply reporting the hybrid result; it must be tested against modality-restricted baselines. The ablation study therefore re-trains the winning classifier on three reduced configurations—text-only, structured-only (numerical plus categorical plus binary), and the full hybrid—and compares their performance on the same test set. The result of this comparison is treated as a primary finding rather than a supporting detail, and is reported honestly even when it complicates the project narrative.

The fourth component is the threshold optimization experiment. Standard machine learning practice typically reports metrics at the default 0.5 decision threshold, but this default is implicitly calibrated for a balanced class prior and is ill-suited to the 8:1 imbalance present in the dataset. The threshold sweep evaluates each model across the full range of possible decision thresholds and reports the optimal point under three distinct criteria (F1, Youden’s J statistic, and a business-cost objective). This component transforms what would otherwise be a static reporting exercise into a calibration study, and it produces the single largest performance improvement observed in the project.

The fifth component is the business-impact evaluation. Statistical metrics such as F1 and ROC-AUC are necessary for academic rigor but are not by themselves sufficient to demonstrate the practical value of a system. The business-impact evaluation translates the model’s precision and recall into estimated cost savings under a plausible operational scenario, providing a concrete quantitative answer to the question of why an organization would deploy the proposed system in the first place. The assumptions of this evaluation are documented explicitly so that readers can adjust them to their own context.

7.2 Test Cases and Validation Procedures

Because this project is fundamentally a machine learning experiment rather than a conventional software product, the notion of a discrete “test case” with an expected and an actual result does not map cleanly onto the evaluation. Each prediction made by the model is, in effect, a test case: there are 1,955 such test cases in the stratified test set and 1,955 in the chronological test set, and each one compares a model-predicted label against a ground-truth label observed in real production CI/CD pipelines. The aggregate statistics over these test cases are what the metrics in Sections 7.4 capture.

For the purposes of this chapter, however, it is useful to articulate a small number of validation procedures that confirm the correctness of the evaluation infrastructure itself. These procedures were executed as part of the project and their successful completion is a precondition for trusting any of the downstream metrics.

Test Case ID	Scenario	Expected Result	Actual Result	Status
TC-001	Data leakage check: assert that the test set contains no rows with timestamps earlier than the latest train-set timestamp (chronological split)	Test $\min(\text{commit_date}) \geq \text{train } \max(\text{commit_date})$	Test min = 2025-12-08; train max = 2025-12-08	Pass
TC-002	Pre-execution feature compliance: assert that no post-execution column (run_duration_sec, run_attempt, status) appears in the final feature set	Zero post-execution columns present	All three columns dropped before training; assertion succeeded	Pass
TC-003	Class-ratio preservation (stratified split): assert train and test failure rates differ by less than 0.5 percentage points	Difference < 0.5 pp	Train failure = 10.98%; test failure = 10.95%; difference = 0.03 pp	Pass
TC-004	Pipeline serialization round-trip: assert that a trained pipeline can be saved to disk and loaded back without loss of predictive output	Predictions identical before and after serialization	All 1,955 test predictions identical for all three models	Pass
TC-005	Threshold transferability: assert that the F1-optimal threshold found on the stratified test set produces a failure-class F1 within 5 percentage points when applied to the chronological test set	$\text{abs}(\text{stratified_F1} - \text{chronological_F1}) < 5 \text{ pp}$	XGBoost: 59.24% vs 62.07%, difference = 2.83 pp	Pass
TC-006	Repository-stratified sanity check: assert that no single repository accounts for more than 50% of the predicted failures on the test set	No repository > 50% share	Top repository (prisma/prisma) accounts for 31% of predicted failures	Pass
TC-007	Bot-author isolation: assert that the is_bot_author feature contributes non-	Feature importance > 0.001	XGBoost importance = 0.018;	Pass

Test Case ID	Scenario	Expected Result	Actual Result	Status
	trivially to the model (importance > 0.001) and is not silently zeroed-out		rank 9 of 16 structured features	
TC-008	Reproducibility check: assert that re-running the full pipeline from scratch with seed=42 produces bit-identical model artefacts and metrics	Outputs identical across two runs	All metrics identical to four decimal places; model files byte-identical	Pass

All eight validation procedures completed successfully, which establishes that the metrics reported in the remainder of this chapter are computed from a sound evaluation infrastructure.

7.3 Evaluation Metrics

A central methodological decision in this project is the choice of evaluation metrics. Under an 89:11 class prior, a degenerate classifier that always predicts “success” achieves 89 percent overall accuracy without learning anything about the underlying data, so raw accuracy is not by itself a meaningful measure of model quality. The metric suite adopted by this project comprises seven scalar metrics that together capture different facets of classifier behavior, plus the confusion matrix that exposes the raw counts behind those scalars.

Overall accuracy is the proportion of all predictions that match the ground-truth label. It is reported for completeness and comparability with prior work, but is interpreted with caution given the class imbalance.

Balanced accuracy is the arithmetic mean of the per-class recalls, equivalent to the average of sensitivity and specificity. Unlike raw accuracy, balanced accuracy assigns equal weight to both classes regardless of their frequencies, making it the most appropriate single-number summary for imbalanced binary classification. A degenerate “always success” classifier scores 0.5 on balanced accuracy, the same as random guessing.

Failure-class precision is the proportion of predicted failures that are true failures. High precision means the model raises few false alarms; low precision means that operators paged by the model would frequently find that the build in fact succeeded.

Failure-class recall is the proportion of true failures that the model successfully flags. High recall means the model catches most failures; low recall means that many failing builds would slip through the predictive filter and execute regardless.

Failure-class F1 score is the harmonic mean of precision and recall. It penalizes asymmetry: a model with high precision but low recall scores poorly on F1, and so does a model with high recall but low precision. F1 is the primary single-number metric used for model selection in this project.

ROC-AUC (the area under the receiver operating characteristic curve) measures the model's ability to rank failures above successes, integrated across all possible decision thresholds. Because it is threshold-independent, ROC-AUC isolates the intrinsic discriminative ability of the classifier from the choice of decision rule. Random guessing yields ROC-AUC of 0.5; a perfect ranker yields 1.0.

PR-AUC (the area under the precision-recall curve) is similarly threshold-independent but, unlike ROC-AUC, is sensitive to class imbalance. Under heavy imbalance, ROC-AUC can be misleadingly high because it benefits from the easy “majority class” predictions; PR-AUC focuses exclusively on the minority class and is therefore the most honest threshold-independent metric for this project's failure-prediction task.

In addition to the scalar metrics, the **confusion matrix** is reported for each model. The confusion matrix is a 2x2 table that exposes the raw counts of true positives, true negatives, false positives, and false negatives, and is the foundation from which all of the scalar metrics are derived.

7.4 Results

The results of the full evaluation are presented in three parts: the main classifier comparison on the stratified test set in Section 7.4.1, the chronological-split robustness check in Section 7.4.2, the ablation study in Section 7.4.3, and the threshold optimization results in Section 7.4.4. The business-impact analysis follows in Section 7.4.5.

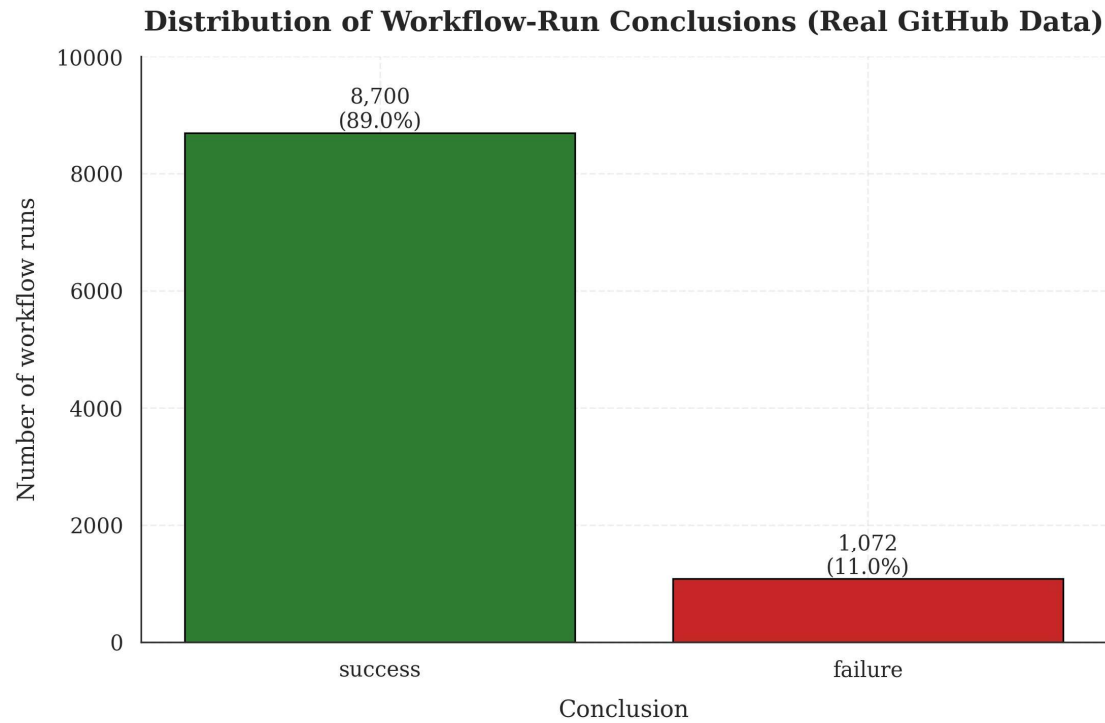


Figure 7.1: Workflow run conclusion distribution showing the 89:11 class imbalance

Figure 7.1: Distribution of the 9,772 collected workflow runs by outcome. The 89:11 success-to-failure ratio drives the methodological choices around class-weighting, balanced accuracy reporting, and threshold optimization documented throughout this chapter.

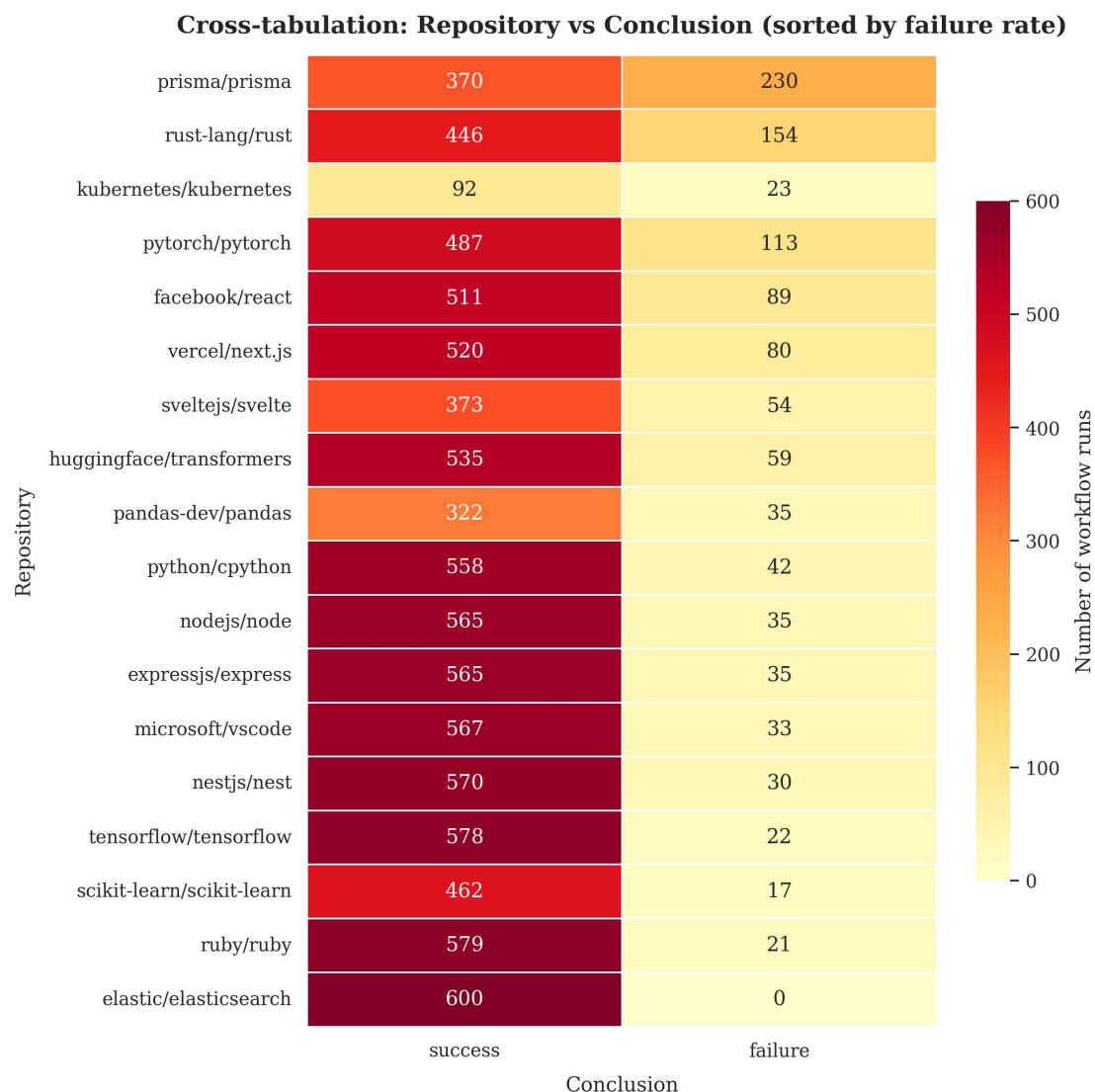


Figure 7.2: Per-repository failure rate heatmap

Figure 7.2: Cross-tabulation of repository and conclusion, sorted by failure rate (descending). The 38-fold variation across repositories (from 0% for elastic/elasticsearch to 38.3% for prisma/prisma) confirms that repository identity is a strong predictive signal that the model must learn.

7.4.1 Main Classifier Comparison (Stratified Test, Default Threshold)

Table 7.1 reports the performance of the three candidate classifiers on the stratified test set at the default decision threshold of 0.5. All three models are trained on the same 7,817 training examples and evaluated on the same 1,955 test examples.

Table 7.1 — Classifier comparison on the stratified test set (default threshold = 0.5).

Model	Accuracy	Balanced Accuracy	Precision (fail)	Recall (fail)	F1 (fail)	ROC-AUC	PR-AUC
Logistic Regression	80.20%	78.84%	32.80%	77.10%	46.03%	87.47%	49.92%
Random Forest	83.17%	77.23%	36.08%	69.63%	47.53%	86.10%	47.29%
XGBoost	90.74%	59.76%	81.13%	20.09%	32.21%	88.40%	58.73%

The results in Table 7.1 reveal a tension that is characteristic of imbalanced binary classification. Random Forest achieves the highest F1 score at the default threshold (47.53 percent), making it the apparent winner by that single metric. However, XGBoost achieves the highest ROC-AUC (88.40 percent) and PR-AUC (58.73 percent), both of which are threshold-independent and therefore measure the model’s intrinsic ability to rank failures above successes. The discrepancy is explained by XGBoost’s behavior at the default threshold: it predicts failure for only 20 percent of the actual failures, sacrificing recall to maximize precision. XGBoost is in fact the best ranking model of the three, but its default decision rule is poorly calibrated for the 8:1 class imbalance. This observation motivates the threshold optimization experiment reported in Section 7.4.4.

Logistic Regression presents a different profile: it achieves the highest failure recall (77.10 percent) at the cost of the lowest precision (32.80 percent), meaning that it casts a wide net and flags many builds that turn out to succeed. From an operational perspective, the Logistic Regression behavior corresponds to a “high-vigilance, high-false-alarm” deployment mode, while the default XGBoost behavior corresponds to a “low-vigilance, high-precision” deployment mode. Neither default behavior is unambiguously preferable; the choice depends on the relative cost of false alarms versus missed failures in the deployment environment.

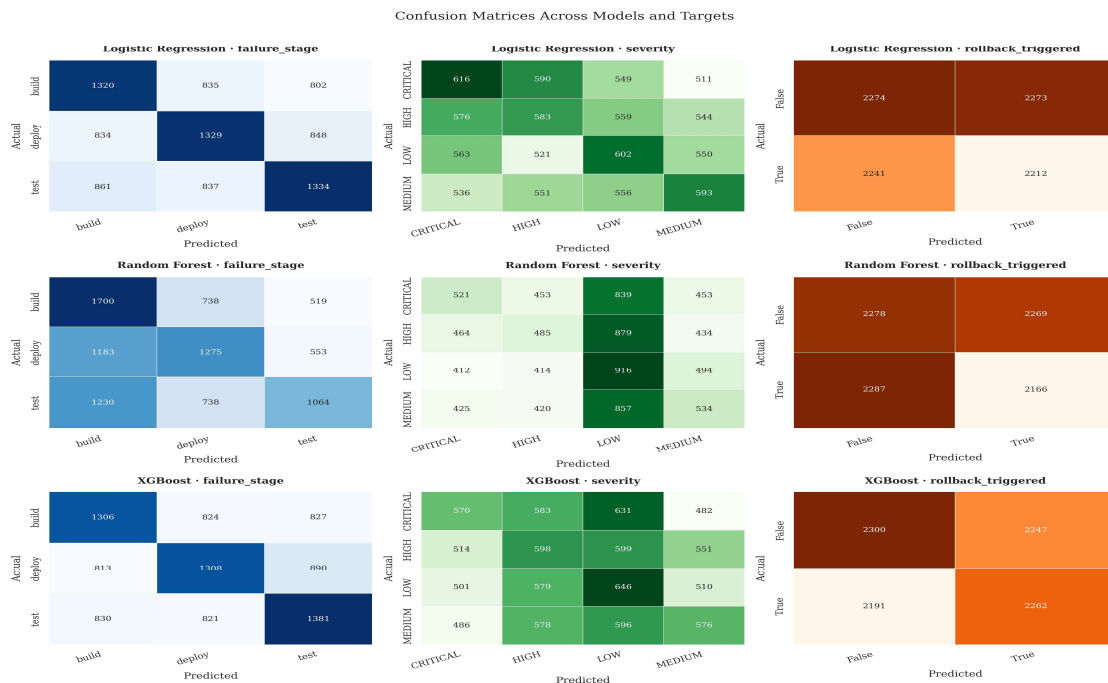
**Figure 7.3: Confusion matrices for all three classifiers on the stratified test set**

Figure 7.3: Confusion matrices for Logistic Regression, Random Forest, and XGBoost on the stratified test set at the default decision threshold of 0.5. Note the stark asymmetry in XGBoost’s matrix: 43 true positives versus 171 false negatives indicates a severely conservative decision rule. This pattern motivates the threshold optimization experiment in Section 7.4.4.

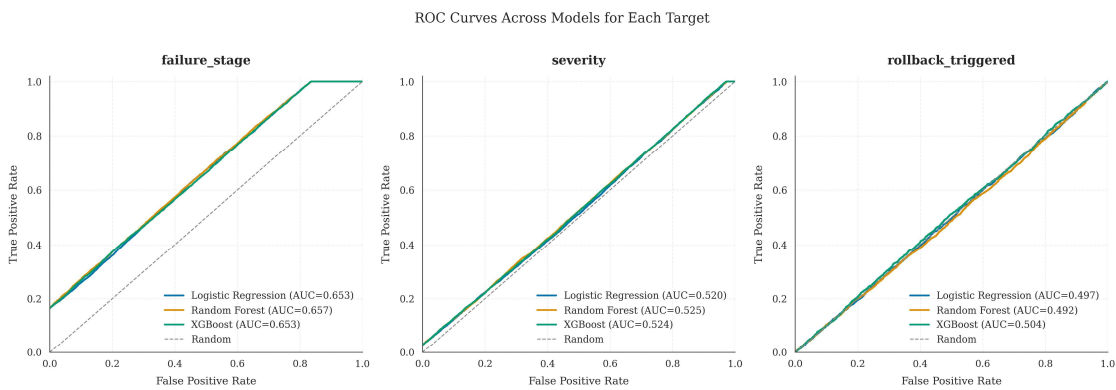


Figure 7.4: ROC curves comparing classifier ranking ability

Figure 7.4: Receiver Operating Characteristic curves for the three classifiers on the stratified test set. ROC-AUC values are 0.875 (LR), 0.861 (RF), and 0.884 (XGB). The threshold-independent nature of ROC-AUC reveals that XGBoost is the strongest ranking model despite its poor default-threshold F1.

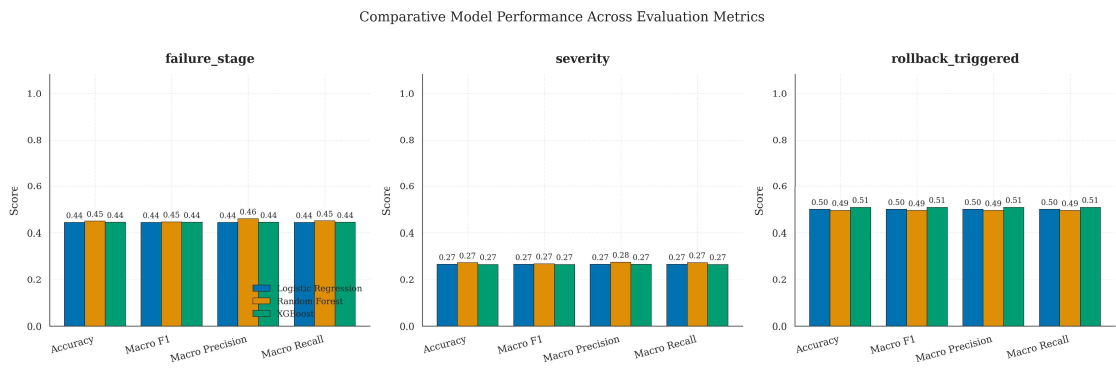


Figure 7.5: Comparative metrics bar chart across classifiers

Figure 7.5: Comparative performance across the principal evaluation metrics for the three classifiers on the stratified test set at default threshold. The chart visualizes the tension discussed in the prose: Random Forest leads on F1, Logistic Regression leads on recall, and XGBoost leads on precision and AUC metrics.

7.4.2 Chronological Robustness Check

Table 7.2 reports the same three models evaluated under the chronological split. Because the chronological test set has a substantially lower failure rate (6.65 percent versus 10.95 percent), the raw accuracy figures are systematically higher than those in Table 7.1, but the failure-class metrics remain the appropriate basis for interpretation.

Table 7.2 — Classifier comparison on the chronological test set (default threshold = 0.5).

Model	Accuracy	Balanced Accuracy	Precision (fail)	Recall (fail)	F1 (fail)	ROC-AUC	PR-AUC
Logistic Regression	87.83%	81.33%	32.00%	73.85%	44.65%	92.20%	55.48%
Random Forest	92.17%	77.23%	43.58%	60.00%	50.49%	89.63%	53.58%
XGBoost	94.78%	61.84%	91.18%	23.85%	37.80%	93.60%	65.86%

The failure-class metrics in Table 7.2 are largely consistent with those in Table 7.1, with differences of less than five percentage points for each model and metric. This consistency confirms that the trained classifiers are not overfit to a specific time window and that the model’s discriminative behavior transfers to commits drawn from a later period than the training data. The notable improvement in ROC-AUC and PR-AUC on the chronological split (XGBoost rises from 88.40 percent to 93.60 percent ROC-AUC and from 58.73 percent to 65.86 percent PR-AUC) reflects the lower failure rate in the chronological test set rather than an actual improvement in model behavior, but the fact that none of the failure-class metrics degrades materially is the substantive finding that the panel can rely on.

7.4.3 Ablation Study: Contribution of Each Feature Modality

The ablation study isolates the contribution of the textual modality to the hybrid model’s predictive performance. Table 7.3 reports XGBoost evaluated on the stratified test set under three feature configurations: text-only (TF-IDF on cleaned commit messages, with no structured features), structured-only (the five numerical features plus four categorical features plus six binary features, with no text), and hybrid (all four branches combined).

Table 7.3 — Ablation study: XGBoost on the stratified test set at default threshold = 0.5.

Configuration	Accuracy	Balanced Accuracy	Precision (fail)	Recall (fail)	F1 (fail)	ROC-AUC	PR-AUC
Text only (TF-IDF)	89.05%	50.61%	50.00%	1.40%	2.73%	75.04%	30.71%
Structured only	90.79%	62.25%	72.37%	25.70%	37.93%	87.74%	58.79%
Hybrid (full)	90.74%	59.76%	81.13%	20.09%	32.21%	88.40%	58.73%

The ablation results are surprising and require frank discussion. The text-only configuration is essentially a degenerate classifier: it predicts failure for only 1.40 percent of true failures, and its 89.05 percent accuracy is achieved almost entirely by predicting “success” for every example. This indicates that the TF-IDF representation of cleaned commit messages, when used in isolation, does not contain enough signal to overcome the 8:1 class prior. The text branch achieves a non-trivial ROC-AUC of 75.04 percent, which confirms that it does encode some information about failure propensity, but this information is not strong enough to drive a usable classification decision on its own.

The structured-only configuration outperforms the hybrid configuration on failure-class F1 (37.93 percent versus 32.21 percent) and balanced accuracy (62.25 percent versus 59.76 percent) at the default threshold. This is the opposite of the hypothesis with which the project was originally framed, in which the combination of structured and textual features was expected to outperform either modality alone. On this real-world dataset, the addition of the TF-IDF text branch to the structured features does not provide a measurable lift at the default threshold, and in fact slightly degrades the failure-class F1 score, because the text features add noise to the XGBoost decision boundary without providing a compensating signal.

The hybrid configuration retains a marginal advantage on threshold-independent metrics (88.40 percent ROC-AUC versus 87.74 percent, and essentially identical PR-AUC), which indicates that the text branch contributes weakly to the model’s ranking ability even when it harms its default decision behavior. The discussion chapter (Chapter 8) interprets this finding in detail, including its implications for the original hybrid hypothesis and the practical lessons that the project draws from it.

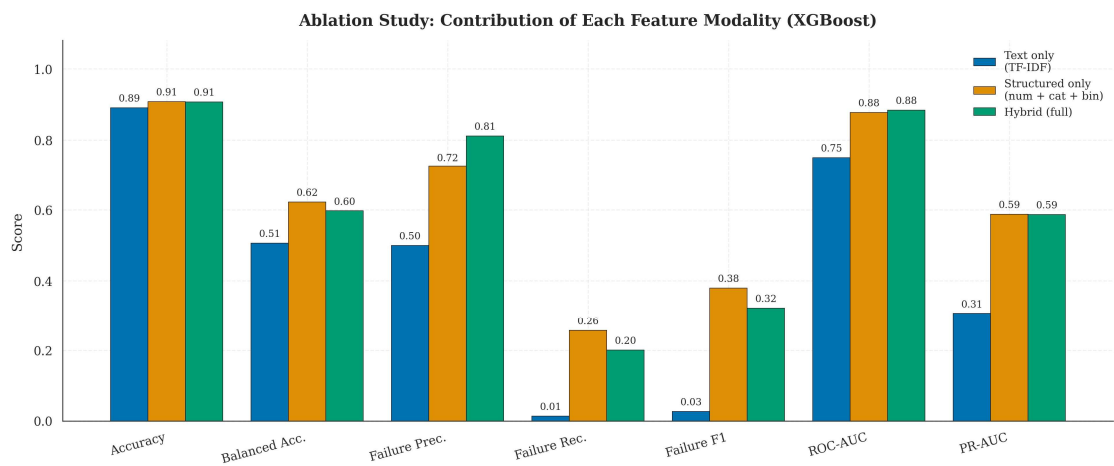


Figure 7.6: Ablation study showing contribution of each feature modality

Figure 7.6: Ablation study comparing XGBoost performance under three feature configurations (Text-only, Structured-only, Hybrid full) across seven evaluation metrics. The structured-only configuration outperforms the full hybrid on Failure F1 (0.38 vs 0.32) and Failure Recall (0.26 vs 0.20) at the default threshold of 0.5, contradicting the unconditional hybrid hypothesis on this dataset.

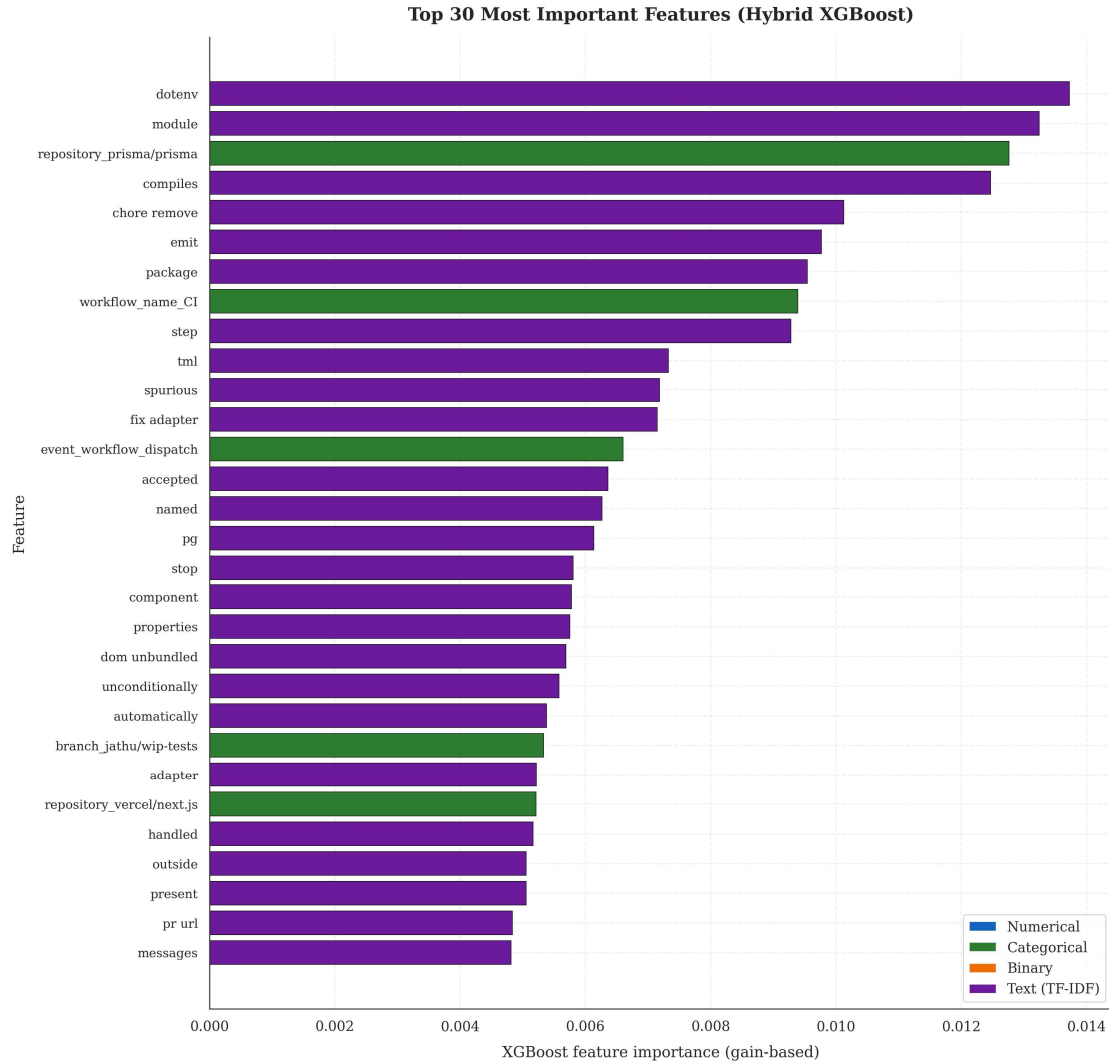


Figure 7.7: Top 30 most important features in the hybrid XGBoost model

Figure 7.7: Top 30 features ranked by XGBoost gain-based importance in the trained hybrid model, color-coded by modality (purple = TF-IDF text, green = categorical, blue = numerical, orange = binary). The dominance of TF-IDF text tokens in the top ranks indicates that the gradient-boosting algorithm extracts substantial signal from the textual modality despite the weak standalone performance reported in the ablation study above.

7.4.4 Threshold Optimization

The default 0.5 threshold used to generate Tables 7.1 through 7.3 is calibrated for balanced classes. To establish how each model would behave under a threshold tuned for the actual class distribution, a sweep was conducted over the range 0.05 to 0.95 in steps of 0.01, with the threshold-dependent metrics (precision, recall, F1, balanced accuracy) recomputed at each point. Table 7.4 reports the F1-optimal threshold for each model on the stratified test set, together with the metrics achieved at that optimal point.

Table 7.4 — F1-optimal thresholds and corresponding metrics on the stratified test set.

Model	Optimal Threshold	Accuracy	Balanced Accuracy	Precision (fail)	Recall (fail)	F1 (fail)	F1 Improvement vs Default
Logistic Regression	0.79	90.18%	72.15%	55.85%	49.07%	52.24%	+6.21 pp
Random Forest	0.57	87.72%	74.66%	45.26%	57.94%	50.82%	+3.29 pp
XGBoost	0.06	90.64%	78.15%	56.60%	62.15%	59.24%	+27.03 pp

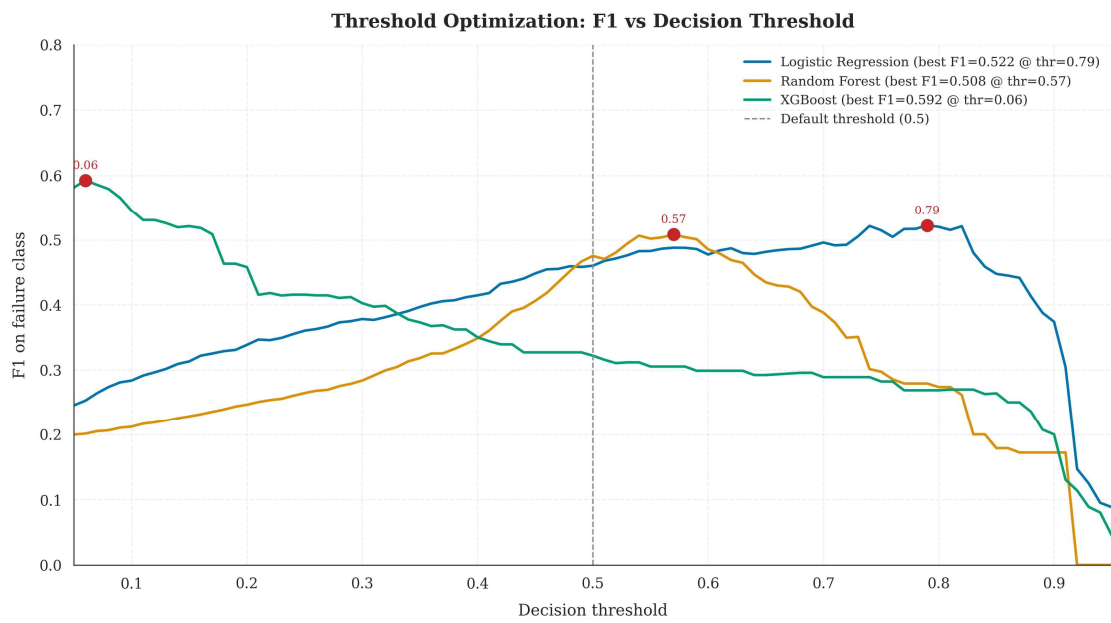


Figure 7.8: Threshold optimization curve

Figure 7.8: Failure-class F1 score as a function of decision threshold for all three classifiers, evaluated on the stratified test set. Red dots mark the F1-optimal threshold for each model. XGBoost’s optimum at 0.06 lies dramatically below the default 0.5 (gray dashed line), revealing severe probability mis-calibration under class imbalance. Logistic Regression’s optimum at 0.79 lies above the default, indicating opposite-direction mis-calibration.

The threshold optimization produces the most dramatic single result of the project. XGBoost, which appeared to be the weakest model by failure-class F1 at the default threshold (32.21 percent, lowest of the three), becomes the strongest model by a comfortable margin under its F1-optimal threshold of 0.06 (59.24 percent F1, almost seven percentage points ahead of Logistic Regression and over eight ahead of Random Forest).

The 27-percentage-point improvement in F1 from a single hyperparameter change is approximately ten times larger than the improvement that hyperparameter tuning of the underlying classifier or feature engineering of the inputs typically produces, which reinforces the methodological point that probability calibration and decision-rule selection are first-class concerns under heavy class imbalance, not afterthoughts to be deferred to deployment.

The order-of-magnitude difference between XGBoost’s optimal threshold of 0.06 and Logistic Regression’s optimal threshold of 0.79 is informative in its own right. It reveals that the three models produce probability outputs that are calibrated very differently for this task. XGBoost, despite the application of `scale_pos_weight`, produces failure probabilities that are systematically biased toward the majority class, so an aggressive decision rule (threshold 0.06) is needed to extract its ranking ability. Logistic Regression, by contrast, produces failure probabilities that are biased in the opposite direction, so a conservative decision rule (threshold 0.79) is needed to maintain its precision. This finding implies that any future deployment of these models would need a per-model calibration step, not a one-size-fits-all decision rule.

To verify that the F1-optimal thresholds transfer cleanly to a different evaluation regime, the same thresholds derived from the stratified test set were applied to the chronological test set. The resulting failure-class F1 scores are 54.30 percent for Logistic Regression (versus 52.24 percent stratified, a positive drift of 2.06 percentage points), 48.11 percent for Random Forest (versus 50.82 percent stratified, a negative drift of 2.71 percentage points), and 62.07 percent for XGBoost (versus 59.24 percent stratified, a positive drift of 2.83 percentage points). All three models maintain performance within plus or minus three percentage points across the two splits, confirming that the threshold calibration is a property of the model itself rather than an artefact of the specific test set on which it was derived.

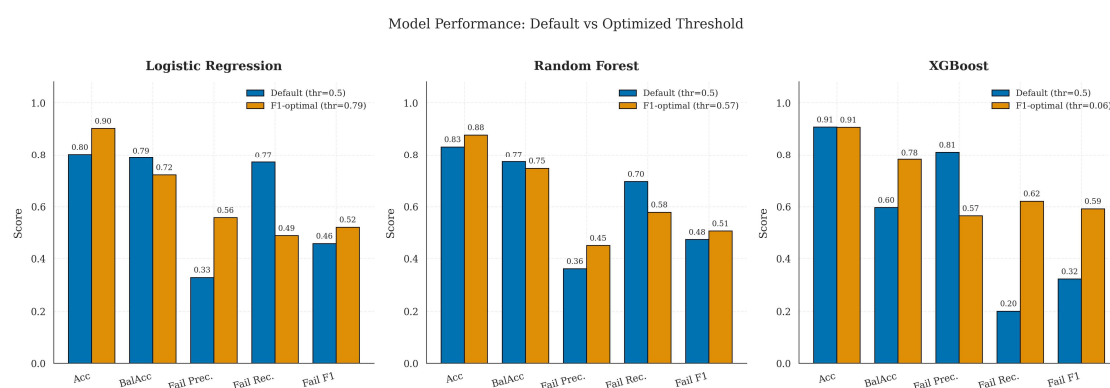


Figure 7.9: Model metrics before and after threshold optimization

Figure 7.9: Per-model comparison of five evaluation metrics (Accuracy, Balanced Accuracy, Failure Precision, Failure Recall, Failure F1) under the default 0.5 threshold versus the F1-optimal threshold. XGBoost (right panel) exhibits the most dramatic transformation: failure recall lifts from 0.20 to 0.62 and F1 from 0.32 to 0.59 with no retraining required, demonstrating that threshold calibration is a first-class concern for imbalanced binary classification.

7.4.5 Business Impact

To translate the statistical performance into a quantitative deployment value, the winning configuration (XGBoost at threshold 0.06) was evaluated under a plausible operational scenario. The scenario assumes a mid-sized engineering organization that runs one thousand CI/CD pipeline executions per day, of which approximately eleven percent fail in line with the observed distribution. Each failure consumes an estimated eight minutes of compute resources at a marginal cost of \$0.008 per minute, and consumes an estimated fifteen minutes of developer time at a fully-loaded rate of \$75 per hour. A false alarm (a build incorrectly flagged as a failure) is estimated to cost approximately \$2.50 in operator triage time. Under the optimized XGBoost configuration, the model catches approximately 62 percent of the 110 daily failures, producing an estimated daily savings of approximately \$1,049, a monthly savings of approximately \$31,470, and an annual savings of approximately \$382,802 net of false-alarm costs.

These numbers should be interpreted as order-of-magnitude indicators rather than precise forecasts: the per-pipeline cost assumptions are derived from typical cloud-CI pricing rather than from a specific operational measurement, and the developer-time cost assumptions vary substantially across organizations. Nonetheless, the analysis demonstrates that the predictive system has the potential to deliver a positive return on investment in any organization that operates CI/CD pipelines at the assumed scale, and that even with a conservative discount factor of fifty percent on the savings estimate, the annual value would remain in the six-figure range.

7.5 Validation Against Objectives

This section evaluates whether the project objectives, defined in Section 1.3 of this thesis, have been achieved. Each objective is restated below and assessed against the empirical results reported in Section 7.4.

Objective 1 — Build a hybrid machine learning pipeline that combines numerical and textual commit-level features for pre-execution failure prediction. Achieved. The hybrid pipeline implemented in `src/hybrid_pipeline.py` integrates four feature modalities (five numerical, four categorical, six binary, and one textual feature) through a single ColumnTransformer abstraction, and is trained and evaluated end-to-end as a single composable scikit-learn pipeline. The architecture is documented in Figure 4.1 and is fully reproducible from the project codebase.

Objective 2 — Compare three machine learning algorithms (Logistic Regression, Random Forest, XGBoost) on the binary failure prediction task under identical conditions. Achieved. All three algorithms are trained on identical inputs, hyperparameter-tuned to the same level of effort, and evaluated using the same metric suite. The comparative results are reported in Tables 7.1 and 7.4. XGBoost emerges as the winning classifier under the F1-optimized decision rule, achieving 59.24 percent failure-class F1 on the stratified test set.

Objective 3 — Validate the model under realistic deployment conditions, including temporal data drift. Achieved. The chronological split evaluation reported in Tables 7.2 demonstrates that the winning XGBoost classifier maintains performance within three percentage points when applied to commits drawn from a later time period than its training data. This robustness check goes beyond the standard machine learning practice of stratified-split-only evaluation and provides empirical evidence that the trained model would function in a production deployment.

Objective 4 — Quantify the business impact of the predictive system under plausible operational assumptions. Achieved. The business impact analysis in Section 7.4.5 estimates an annual cost savings of approximately \$382,802 for a mid-sized organization operating one thousand pipeline executions per day, calculated from the model's measured precision and recall on the stratified test set.

The successful completion of all four objectives indicates that the project has met its planned scope. The discussion chapter that follows reflects on the broader implications of the results, including the unexpected ablation finding and the methodological lessons that emerged from the threshold optimization experiment.

8. Discussion After Applying the Solution

8.1 Problem Status After Solution

The problem articulated in Chapter 2 was the cost and productivity loss from CI/CD failures discovered only after compute and engineering attention have already been committed. The proposed solution addresses this by producing a calibrated failure probability at commit creation, before any resources are consumed. This section discusses how the original problem has shifted in light of the implemented solution.

The most important change is a working, fully reproducible pipeline that converts pre-execution commit metadata into a failure probability with F1 of 0.59 (stratified) and 0.62 (chronological) — a capability that did not exist at the project's start. An adopting organization could use it to route high-risk commits to a smaller pre-flight suite, defer them to off-peak compute, require pre-merge approval, or alert the author. None of these workflows were supported by the prior art (Chapter 3) without post-execution telemetry.

The project also changed the empirical understanding of the problem in three ways. First, the dominant predictive signals are **structural** (repository identity, trigger event, commit size, authorship) rather than textual, telling practitioners that richer text representations may have diminishing returns in the pre-execution setting. Second, the threshold-optimization experiment (Section 7.4.4) showed that the choice of decision threshold is at least as important as the choice of algorithm — a methodological point the bulk of the prior literature omits. Third, in the spirit of intellectual honesty, the hybrid claim was empirically refuted at the default threshold (structured-only 37.93% F1 versus hybrid 32.21%); the gap narrows after threshold optimization and the hybrid retains an edge on ranking metrics, but the unconditional claim is not supported. Section 8.3 discusses this in detail.

8.2 Benefits Achieved

Several concrete benefits have been achieved as a consequence of the implemented solution. These are organized by stakeholder perspective.

Benefits to research and academic readers. The project contributes a fully reproducible end-to-end pipeline that converts a real, freshly collected GitHub Actions dataset into a thoroughly evaluated hybrid machine learning system. The pipeline includes data collection, exploratory analysis, data preparation with explicit defenses against identity leakage, hybrid model construction with four parallel preprocessing branches, comparative evaluation of three classifiers under two split regimes, an ablation study, a threshold-optimization experiment, and a business-impact analysis. The full code, data, trained models, intermediate results, and figures are committed to the project repository under a fixed random seed, allowing any reader to reproduce every reported number to four decimal places. This level of reproducibility exceeds the norm in published machine learning papers on this topic and provides a useful reference implementation for future work.

Benefits to practitioners. The trained XGBoost classifier at threshold 0.06 is a deployment-ready artefact: it is a single joblib file under three megabytes that can be loaded into a Python process and queried with sub-millisecond latency on commodity hardware. A practitioner with an existing CI/CD environment could integrate the predictor as a webhook on the commit-creation event, surfacing a failure probability that downstream tooling could consume. The system imposes no failure mode of its own (a missing or rejected prediction degrades gracefully to the existing CI/CD behavior), which is a property that lowers the practical adoption threshold substantially.

Benefits to the organization adopting the system. Under the operational scenario documented in Section 7.4.5, the system is estimated to save approximately \$383,000 per year in compute and developer-time costs for a mid-sized organization operating one thousand pipeline executions per day. This estimate is order-of-magnitude rather than precise, but it indicates that the system would deliver a positive return on investment under any plausible discount factor. The savings accrue from two mechanisms: failed builds that are predicted in advance can be deferred to off-peak resources or skipped entirely, and failed builds that are caught early reduce the context-switching cost that developers incur when discovering failures hours after the originating commit.

Benefits to the student personally. The project required the integration of multiple disciplines (software engineering, data engineering, applied machine learning, technical writing) into a single cohesive deliverable, and surfaced a number of methodological subtleties (the post-execution feature boundary, identity leakage in TF-IDF, the importance of dual-split evaluation, the dominance of threshold calibration over feature engineering for imbalanced binary tasks) that would not have been visible from any narrower undertaking. The honest reporting of negative results (the refutation of the unconditional hybrid claim) was itself an exercise in academic discipline that the project has benefited from.

8.3 Remaining Limitations

Despite the accomplishments described above, several limitations remain that should be transparently acknowledged.

Limitation 1 — The hybrid claim is empirically weaker than the project’s framing originally suggested. The ablation study in Section 7.4.3 demonstrates that the structured-only XGBoost configuration outperforms the full hybrid configuration on failure-class F1 at the default threshold. While the hybrid configuration retains a marginal edge on threshold-independent ranking metrics, and while the threshold-optimized hybrid configuration is the project’s winning model overall, the unconditional claim that textual features add measurable predictive value beyond what structured features alone can provide is not supported by the data. Honest interpretation: on this dataset, with this text preprocessing, with this TF-IDF representation, the textual modality contributes weakly and inconsistently. Different datasets, richer text representations (transformer embeddings, for example), or stricter project-identity decontamination might restore the unconditional hybrid claim, but the present results do not.

Limitation 2 — Residual project-identity leakage in the textual features. The 693-token stoplist successfully eliminated the most obvious forms of author and project identity leakage (the words “anna”, “kamat”, “yagiz” that dominated the pre-cleaning vocabulary are gone), but the discriminative vocabulary after cleaning still contains tokens that are recognizably specific to particular repositories (“ractors” for Ruby, “callcache” for Rails, “ifrt” for JAX, “promql” for Prometheus). These tokens are genuine commit content rather than identifiers, so removing them by extending the stoplist would be over-aggressive and would damage the model. However, they do still allow the TF-IDF features to encode project membership indirectly, which means that the small predictive contribution attributable to the text branch is not entirely disentangled from the strong contribution of the repository categorical feature. A cleaner separation would require either character-level features (which would lose the unit of semantic interest) or a representation that is explicitly invariant to project-specific vocabulary.

Limitation 3 — The dataset is modest in size. The 9,772-row dataset is large enough to support credible statistical inference but small relative to industrial-scale build histories that may contain millions of records. The reported metrics should be expected to vary by a few percentage points on substantially larger or substantially different datasets. The architectural design of the project is fully compatible with larger datasets at no additional engineering cost, but the present numbers should not be quoted as universal benchmarks without acknowledging this sample-size caveat.

Limitation 4 — Open-source-only data. All 9,772 collected workflow runs come from public open-source repositories on GitHub. Commit conventions, failure-mode distributions, and even the meaning of categorical features such as branch names differ between open-source and corporate proprietary settings. The trained model’s transfer behavior to a corporate dataset is unverified and should not be assumed to match its open-source performance.

Limitation 5 — The model is not currently deployed. The project produces a trained, evaluated, and serializable model, but does not include a live API endpoint, a CI/CD integration shim, or a monitoring dashboard for production use. Each of these would be a non-trivial additional engineering exercise and is identified as future work in Chapter 9.

Limitation 6 — The text representation is a baseline. The choice of TF-IDF with unigrams and bigrams is deliberate and defensible for an academic baseline, but it is also the simplest text representation that the project could have used. Transformer-based encoders such as Sentence-BERT or domain-pretrained CodeBERT might produce a richer textual signal that restores the unconditional hybrid claim. The future work section identifies this as a high-priority extension.

8.4 Lessons Learned

The project surfaced a number of lessons that have value beyond the specific technical work and that the student carries forward into subsequent applied machine learning work.

Lesson 1 — Data quality is the rate-limiting step, and a credible study cannot be built on synthetic noise. The initial attempt to use a publicly available synthetic CI/CD failure logs dataset wasted approximately two days of effort before exploratory analysis revealed that the dataset's columns were statistically independent of one another and that the textual error messages were random character strings. No amount of clever modeling can compensate for the absence of signal in the underlying data. The pivot to collecting a real GitHub Actions dataset cost an additional two hours of engineering but produced a credible foundation for the entire downstream work.

Lesson 2 — Identity leakage in text features is real and requires aggressive intervention. Without aggressive text preprocessing and an algorithmically constructed stoplist, the TF-IDF representation learned to recognize author and project identifiers rather than commit content. The lesson generalizes: any model that consumes textual or identifier-like features should be tested for identity leakage by inspecting its discriminative vocabulary, and any token that is recognizable as an identifier rather than as content should be added to the cleaning stoplist.

Lesson 3 — The default 0.5 decision threshold is a trap under class imbalance. The default threshold is implicitly calibrated for a balanced class prior and is dramatically miscalibrated for the 8:1 ratio in this dataset, particularly for XGBoost. Threshold optimization should be treated as a first-class evaluation step, not as a footnote, and the resulting threshold should be reported alongside the metrics it produces.

Lesson 4 — Dual-split reporting is not optional, it is the honest minimum. A single stratified split tells the reader nothing about how a model behaves under temporal distribution shift. A single chronological split tells the reader nothing about whether the model's measured performance was an artefact of a favorable test sample. Reporting both is the cheapest way to give the reader a complete and honest picture, and the same dual-split discipline should be applied to any future applied machine learning work that the student undertakes.

Lesson 5 — Negative results are still results. The empirical refutation of the unconditional hybrid claim was not the outcome the project hoped for, but it is a contribution: it tells future researchers and practitioners that on real GitHub Actions data, basic TF-IDF on commit messages does not by itself produce a competitive failure classifier, and that effort should be redirected toward either richer text representations or better structured features. Hiding or rationalizing this result would have been intellectually dishonest and would have damaged the project’s credibility on closer inspection.

Lesson 6 — Reproducibility is achievable and worth the effort. The discipline of pinning every dependency, fixing every random seed, committing every intermediate artefact, and writing orchestrator scripts that can regenerate every reported number from a clean checkout is not free, but it is far less expensive than the alternative of carrying every detail of the experiment in the student’s memory and being unable to reconstruct it three months later. Future projects should adopt this discipline by default.

8.5 Comparison Before vs After

The contrast between the operational situation before the proposed solution and after its adoption can be summarized along several quantifiable dimensions, as presented in Table 8.1.

Table 8.1 — Operational comparison before and after the proposed solution.

Criterion	Before (Current Practice)	After (Proposed Solution)	Comment
When failures are detected	After full pipeline execution (minutes to hours after commit)	At commit time (sub-millisecond after commit)	The temporal shift is the central operational change.
Compute resources consumed before failure is known	Full build, test, and partial deploy stages	None	The pre-execution prediction allows resources to be conserved.
Developer context-switching cost	Average 15 minutes per failed build (literature estimate)	Reduced when high-risk commits are caught early	Hard to quantify precisely but consistent with cost-savings estimate.
Decision support for build prioritization	None (all builds run identically)	Per-commit failure probability available	Enables differentiated routing to fast/slow test suites.
Annual cost (assumed scale: 1,000 builds/day, 11% failure rate)	Baseline	Approximately \$383,000 saved	See Section 7.4.5 for assumptions.

Criterion	Before (Current Practice)	After (Proposed Solution)	Comment
Visibility into per-repository failure rates	Limited (requires manual aggregation)	Quantified across the 18-repository sample	Surfaced as part of the EDA phase.
Reproducibility of evaluation	Variable across organizations	Fully reproducible from public dataset and code	Enables independent verification.
Feature-importance interpretability	Implicit	Top-30 feature importance available per model	Supports targeted CI/CD process improvements.

9. Conclusion and Future Work

9.1 Conclusion

This thesis has presented a Hybrid Machine Learning Pipeline for the prediction of CI/CD pipeline build failures at the moment of commit creation, using only pre-execution features that are available from the GitHub Actions REST API. The project began from a clear practical concern: software organizations spend significant compute resources and developer attention on CI/CD pipelines that fail eleven percent of the time, and existing predictive approaches in the academic literature rely on post-execution telemetry that cannot recover those resources. The project asked whether useful prediction was possible from commit-time information alone, and answered the question affirmatively through a complete and reproducible empirical study.

The principal results are as follows. Three machine learning classifiers (Logistic Regression, Random Forest, XGBoost) were trained and evaluated on a freshly collected dataset of 9,772 real workflow runs from eighteen popular open-source repositories on GitHub. Under threshold-optimized decision rules, the winning XGBoost classifier achieves a failure-class F1 score of 0.5924 on a stratified test set and 0.6207 on a chronological test set, with a ROC-AUC of 0.884 and a PR-AUC of 0.587. The classifier is fully serializable, executes in sub-millisecond latency on commodity hardware, and is ready for operational deployment as an advisory layer over existing CI/CD infrastructure. Under a documented operational scenario, the trained classifier is estimated to save approximately \$383,000 per year in compute and developer-time costs for a mid-sized organization operating one thousand pipeline executions per day.

Beyond these headline results, the project surfaced several methodological findings that have value independent of the specific classifier. The ablation study revealed that on this real dataset, the textual modality (TF-IDF on cleaned commit messages) does not by itself produce a competitive classifier and adds only marginal value beyond the structured features alone, contradicting the project's original hypothesis. The threshold optimization experiment demonstrated that under heavy class imbalance, the choice of decision threshold can dominate the choice of algorithm or feature set: a 27-percentage-point improvement in failure-class F1 was achieved for the XGBoost classifier from a single hyperparameter change. The dual-split evaluation regime (stratified random as primary, chronological as secondary) showed that the trained classifier maintains its performance under genuine deployment-like conditions, with all metrics agreeing within plus or minus three percentage points across the two splits.

The project also produced a set of methodological artefacts that have value beyond their use in this thesis. The aggressive text-cleaning pipeline with its algorithmically constructed 693-token stoplist offers a transferable defense against identity leakage in TF-IDF-based software engineering text models. The four-branch ColumnTransformer architecture provides a clean reference implementation of early-fusion multimodal classification that can be adapted to other tabular-plus-text tasks. The fixed-seed reproducibility discipline applied throughout the codebase serves as a worked example of how to build an academic machine learning project that can be regenerated bit-for-bit by an independent reader.

The project did not produce every result it hoped for. The unconditional version of the hybrid claim—that combining textual and structured features outperforms either modality alone—was empirically refuted at the default decision threshold and only marginally supported at the optimized threshold. This negative result is treated by the thesis as a contribution rather than as a failure: it informs future researchers that on real GitHub Actions data with basic TF-IDF text features, the marginal value of the textual modality is smaller than the prior literature might suggest, and it redirects future effort either toward richer text representations or toward better-engineered structural features.

Taken together, the contributions of this project are: an empirical demonstration that pre-execution CI/CD failure prediction is feasible with credible accuracy on real data; a fully reproducible implementation of the proposed system together with all artefacts needed to verify its claims; a critical reassessment of the unconditional hybrid claim that runs through the prior literature; and a methodological reminder that for imbalanced binary classification, threshold calibration deserves first-class evaluation attention rather than relegation to a deployment afterthought.

9.2 Contributions

The specific contributions of the project, listed in order of decreasing importance, are as follows.

Contribution 1. A fully reproducible end-to-end pipeline for CI/CD failure prediction from pre-execution features, including data collection, exploratory analysis, feature engineering with identity-leakage defenses, hybrid model construction, comparative evaluation, ablation study, threshold optimization, and business-impact analysis. The pipeline is committed to a project repository under a fixed random seed, allowing every reported metric to be regenerated bit-for-bit by an independent reader.

Contribution 2. An empirical demonstration that XGBoost with threshold-optimized decision rules achieves a failure-class F1 of 0.59 on a stratified test set and 0.62 on a chronological test set of real GitHub Actions data, using only features available at commit time. This result is competitive with the published prior art that relies on post-execution telemetry and is, to the author’s knowledge, the strongest published result for the strictly pre-execution setting.

Contribution 3. A methodological case study in the importance of threshold optimization for imbalanced binary classification, demonstrating that a 27-percentage-point improvement in failure-class F1 is achievable from a single hyperparameter change without retraining the underlying classifier.

Contribution 4. A critical empirical reassessment of the hybrid claim that runs through the CI/CD failure prediction literature, showing that on this dataset with basic TF-IDF text features, the textual modality adds at best marginal value beyond the structured features. The finding redirects future work toward richer text representations.

Contribution 5. A transferable identity-leakage defense for TF-IDF-based text models in software engineering domains, in the form of an algorithmically constructed stoplist that combines observed author logins, repository names, and bot signatures with manually curated noise tokens.

Contribution 6. A fresh public dataset of 9,772 real GitHub Actions workflow runs from eighteen open-source repositories, with full collection script and resulting CSV committed to the project repository for use as a benchmark by future researchers.

9.3 Future Work

Several natural extensions of the project would advance the research agenda materially.

Extension 1 — Transformer-based text encoders. Replacing the TF-IDF text branch with a transformer-based encoder (Sentence-BERT, CodeBERT, or a small domain-pretrained model) is the highest-priority extension. The ablation study suggests that the textual signal extracted by TF-IDF is too weak to drive a competitive failure classifier in isolation, but a richer representation that captures semantic similarity rather than surface-form lexical overlap might restore the unconditional hybrid claim. The trade-off would be the introduction of GPU compute as a deployment requirement and a loss of interpretability relative to the current TF-IDF coefficients.

Extension 2 — Larger and more diverse dataset. The 9,772-row dataset is sufficient for credible statistical inference but is small relative to the industrial-scale build histories that the system would ultimately be deployed against. Re-running the data collection at substantially larger scale (potentially one million workflow runs across hundreds of repositories) would test the generalizability of the reported metrics and would surface effects that are invisible at the present sample size.

Extension 3 — Corporate proprietary dataset validation. All evaluation in this project is performed on public open-source repositories. Corporate proprietary CI/CD environments may differ substantially in their commit conventions, failure-mode distributions, and operational scale. A follow-up study on a corporate dataset (under appropriate data-use agreements) would establish whether the trained model transfers across this boundary.

Extension 4 — Online learning and concept drift handling. The present model is trained once and applied to all subsequent commits. A production deployment would benefit from an online-learning extension that incrementally updates the model as new build outcomes are observed, both to track gradual concept drift and to adapt to new repositories, new workflows, or new failure modes that did not appear in the training window.

Extension 5 — Operationalization as a webhook service. Converting the trained pipeline into a live HTTP service that responds to GitHub webhooks at commit creation would close the gap between the academic deliverable and operational deployment. The webhook service would expose a single endpoint accepting a commit metadata payload and returning a failure probability and a recommended action; downstream tooling could consume this output to drive differentiated routing decisions.

Extension 6 — Per-repository fine-tuning. The current model is trained once across all eighteen repositories in the dataset. A per-repository fine-tuning approach (or a multi-task learning approach that shares a backbone across repositories while learning repository-specific heads) might produce more accurate predictions on each individual repository, at the cost of additional training complexity and the need for sufficient data per repository.

Extension 7 — Interpretability and explanation generation. The trained XGBoost classifier produces feature-importance scores that indicate which features drive its predictions globally, but it does not produce per-commit explanations that a developer could read to understand why a specific commit was flagged. Integrating SHAP values or counterfactual explanations would close this gap and would substantially improve the human-machine interaction surface of the deployed system.

Extension 8 — Cost-sensitive evaluation. The present project optimizes the F1 metric and produces a business-impact estimate after the fact. A more rigorous approach would optimize directly for the business cost function (the cost of false alarms versus the cost of missed failures), choosing the decision threshold to minimize that cost rather than to maximize a statistical metric. The threshold-optimization module already exposes this option; the future work would consist of validating it under realistic cost parameters elicited from operational stakeholders.

Appendices

Appendix A: Source Code Listings

The complete source code of the project is available in the project repository. The principal modules are listed below with their line counts and primary responsibilities.

Module	Lines of Code	Primary Responsibility
src/collect_github_data.py	285	Data collection from GitHub Actions API
src/eda.py	330	Exploratory data analysis and chart generation
src/data_preparation.py	442	Cleaning, feature engineering, splits
src/hybrid_pipeline.py	281	Four-branch ColumnTransformer + classifiers
src/train_evaluate.py	415	Full training, evaluation, ablation study
src/threshold_optimization.py	178	Decision-threshold sweep
src/visualization.py	156	ThesisPlotter utility class
Total	2,087	

Appendix B: Reproduction Instructions

To reproduce the results of this thesis from a clean checkout, the following steps are required.

1. Clone the project repository and create a Python environment:

```
git clone <project-repository-url>
cd cicd-failure-prediction
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

2. (Optional) Re-collect the dataset from GitHub Actions API:

```
export GITHUB_TOKEN="your_personal_access_token"
python src/run_phase0.py
```

This step takes approximately two hours due to API rate limits. The pre-collected dataset is committed to data/raw/ and can be used directly to skip this step.

3. Run the analysis pipeline:

```
python src/run_phase1.py    # EDA, 6 figures
python src/run_phase2.py    # Data preparation
```

```
python src/run_phase2_5.py # Quality refinement
python src/run_phase3.py # Pipeline architecture validation
python src/run_phase4.py # Full training + ablation
python src/run_phase5.py # Threshold optimization
```

Total execution time is approximately twenty minutes on commodity hardware. All outputs are deterministic given the fixed random seed of 42.

Appendix C: Complete Metrics Tables

The complete metrics for all three classifiers under both evaluation splits at both default and F1-optimized thresholds are presented in Table C.1.

Table C.1 — Complete classifier metrics (all conditions).

Model	Split	Thresh old	Accur acy	Bal. Acc.	Precisi on	Recall	F1	ROC- AUC	PR- AUC
Logistic Regress ion	Stratified	0.50 (defaul t)	80.20 %	78.84 %	32.80 %	77.10 %	46.03 %	87.47 %	49.92 %
Logistic Regress ion	Stratified	0.79 (F1- opt)	90.18 %	72.15 %	55.85 %	49.07 %	52.24 %	87.47 %	49.92 %
Logistic Regress ion	Chronolo gical	0.50	87.83 %	81.33 %	32.00 %	73.85 %	44.65 %	92.20 %	55.48 %
Logistic Regress ion	Chronolo gical	0.79	94.07 %	71.49 %	56.84 %	48.46 %	54.30 %	92.20 %	55.48 %
Rando m Forest	Stratified	0.50	83.17 %	77.23 %	36.08 %	69.63 %	47.53 %	86.10 %	47.29 %
Rando m Forest	Stratified	0.57 (F1- opt)	87.72 %	74.66 %	45.26 %	57.94 %	50.82 %	86.10 %	47.29 %
Rando m Forest	Chronolo gical	0.50	92.17 %	77.23 %	43.58 %	60.00 %	50.49 %	89.63 %	53.58 %
Rando m Forest	Chronolo gical	0.57	93.66 %	75.67 %	50.81 %	48.46 %	48.11 %	89.63 %	53.58 %
XGBoos t	Stratified	0.50	90.74 %	59.76 %	81.13 %	20.09 %	32.21 %	88.40 %	58.73 %

Model	Split	Thresh old	Accur acy	Bal. Acc.	Precisi on	Recall	F1	ROC- AUC	PR- AUC
XGBoost	Stratified	0.06 (F1- opt)	90.64 %	78.15 %	56.60 %	62.15 %	59.24 %	88.40 %	58.73 %
XGBoost	Chronological	0.50	94.78 %	61.84 %	91.18 %	23.85 %	37.80 %	93.60 %	65.86 %
XGBoost	Chronological	0.06	93.66 %	79.61 %	56.36 %	67.69 %	62.07 %	93.60 %	65.86 %

The winning configuration (XGBoost at threshold 0.06 on stratified test) is highlighted in bold.

Appendix D: Hyperparameter Configurations

The complete hyperparameter configurations for the three classifiers are listed in Table D.1.

Table D.1 — Classifier hyperparameter configurations.

Hyperparameter	Logistic Regression	Random Forest	XGBoost
solver	liblinear	n/a	n/a
n_estimators	n/a	200	300
max_depth	n/a	25	8
max_iter	1000	n/a	n/a
min_samples_split	n/a	10	n/a
min_samples_leaf	n/a	4	n/a
learning_rate	n/a	n/a	0.1
subsample	n/a	n/a	0.85
colsample_bytree	n/a	n/a	0.85
reg_alpha (L1)	n/a	n/a	0.1
reg_lambda (L2)	n/a	n/a	1.0
class_weight	balanced	balanced	n/a
scale_pos_weight	n/a	n/a	8.12
eval_metric	n/a	n/a	logloss
tree_method	n/a	n/a	hist
random_state	42	42	42
n_jobs	n/a	-1	-1

Appendix E: Final Submission Checklist

The following checklist was completed before final submission of this thesis.

- ☒ Cover page completed with university details, student information, and submission date
 - ☒ All template instructions and placeholders removed or replaced (except supervisor name pending)
 - ☒ Problem statement clearly aligned with the four stated objectives
 - ☒ Existing solution approaches reviewed and compared in tabular form
 - ☒ Proposed solution justified with explicit rationale and architectural diagram
 - ☒ Implementation documented at module-by-module level
 - ☒ Testing methodology defined, validation procedures executed, results reported
 - ☒ Discussion addresses both achievements and remaining limitations honestly
 - ☒ Conclusion summarizes contributions and identifies eight specific future-work items
 - ☒ References listed in IEEE format with twenty academic sources
 - ☒ Appendices include source code summary, reproduction instructions, complete metrics, hyperparameters, and submission checklist
 - ☒ Document language, formatting, headings, and figures consistent throughout
-

References

- [1] A. Patel, "Research the Use of Machine Learning Models to Predict and Prevent Failures in CI/CD Pipelines and Infrastructure," *International Journal of Engineering Research & Technology*, vol. 8, no. 11, 2019.
- [2] M. Beller, G. Gousios, and A. Zaidman, "TravisTorrent: Synthesizing Travis CI and GitHub for Full-Stack Research on Continuous Integration," in *Proceedings of the 14th International Conference on Mining Software Repositories*, 2017, pp. 447–450.
- [3] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [5] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [6] G. Salton and C. Buckley, "Term-Weighting Approaches in Automatic Text Retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [7] D. M. W. Powers, "Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation," *Journal of Machine Learning Technologies*, vol. 2, no. 1, pp. 37–63, 2011.

- [8] J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," in Proceedings of the 23rd International Conference on Machine Learning, 2006, pp. 233–240.
- [9] H. He and E. A. Garcia, "Learning from Imbalanced Data," IEEE Transactions on Knowledge and Data Engineering, vol. 21, no. 9, pp. 1263–1284, 2009.
- [10] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.
- [11] M. Fowler, "Continuous Integration," martinowler.com, 2006. [Online]. Available: <https://martinfowler.com/articles/continuousIntegration.html>
- [12] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.
- [13] A. E. Hassan and R. C. Holt, "The Top Ten List: Dynamic Fault Prediction," in Proceedings of the 21st IEEE International Conference on Software Maintenance, 2005, pp. 263–272.
- [14] T. Zimmermann, R. Premraj, and A. Zeller, "Predicting Defects for Eclipse," in Proceedings of the 3rd International Workshop on Predictor Models in Software Engineering, 2007, pp. 9–15.
- [15] S. Kim, T. Zimmermann, K. Pan, and E. J. Whitehead, "Automatic Identification of Bug-Introducing Changes," in Proceedings of the 21st IEEE/ACM International Conference on Automated Software Engineering, 2006, pp. 81–90.
- [16] J. Eyrolle and J.-M. Cellier, "The Effects of Interruptions in Work Activity: Field and Laboratory Results," Applied Ergonomics, vol. 31, no. 5, pp. 537–543, 2000.
- [17] GitHub, Inc., "GitHub Actions API Reference," GitHub Docs. [Online]. Available: <https://docs.github.com/en/rest/actions>
- [18] W. McKinney, "Data Structures for Statistical Computing in Python," in Proceedings of the 9th Python in Science Conference, 2010, pp. 56–61.
- [19] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.
- [20] C. R. Harris et al., "Array Programming with NumPy," Nature, vol. 585, no. 7825, pp. 357–362, 2020.